

华中科技大学

课程实验报告

课程名称: 编译技术实验

专业班级: 软件工程 230X 班

学 号: U202317XXX

姓 名: XXX

指导教师: 胡雯蕾

报告日期: 2025 年 12 月 11 日

软件学院

目 录

摘要	I
Abstract	II
1 概述	1
1.1 实验简介	1
1.2 本文组织结构	1
2 语言定义	3
2.1 语言概述	3
2.2 数据类型	3
2.3 变量与数组	4
2.4 运算符	4
2.5 控制语句	6
2.6 函数	7
2.7 注释	8
2.8 文法描述	8
3 词法分析与语法分析	11
3.1 词法分析器设计	11
3.2 语法分析器设计	15
3.3 抽象语法树（AST）设计	17
3.4 错误处理机制	20
3.5 编译流程总结	20
4 符号表管理与语义分析	22
4.1 概述	22

4.2	符号表设计	22
4.3	属性计算	24
4.4	静态语义检查	25
4.5	语义分析流程	29
4.6	错误处理与恢复	30
5	中间代码生成	31
5.1	概述	31
5.2	三地址码设计	31
5.3	临时变量与标号处理	33
5.4	表达式的中间代码生成	34
5.5	语句的中间代码生成	36
5.6	函数的中间代码生成	37
5.7	中间代码示例	37
6	目标代码生成	39
6.1	概述	39
6.2	MIPS 体系结构与寄存器约定	39
6.3	运行时存储组织	40
6.4	内存访问辅助函数	41
6.5	中间代码到 MIPS 指令的翻译	42
6.6	控制流指令翻译	44
6.7	函数调用	44
6.8	目标代码示例	45
7	系统测试	47
7.1	概述	47

7.2	测试用例	48
7.3	完整编译流程演示	49
8	总结与展望	82
8.1	项目总结	82
8.2	技术难点与解决方案	82
8.3	技术亮点	84
8.4	存在的不足与改进方向	86
8.5	心得体会	87
8.6	展望	88
	参考文献	89

1 概述

1.1 实验简介

本实验报告详细介绍了“Mini-C”编译器的实现。它是一个简化的编译器，旨在处理“Mini-C”语言——一种简化版的C语言。整个编译器的设计遵循了编译原理的基本概念和方法，包括词法分析、语法分析、语义分析、中间代码生成、目标代码生成等关键步骤。本报告全面地覆盖了编译器各个模块的具体实现方法和过程，从而为理解编译器的内部工作机制提供了深入的视角。通过对编译器各个模块的深入分析，本报告不仅展示了编译器的内部工作机制，而且为编译原理的学习和理解提供了宝贵的实践经验。

1.2 本文组织结构

本文共八章，具体安排如下：

第一章为概述。

第二章为语言定义，介绍 Mini-C 语言的设计目标、语法特性和语言规范。详细描述支持的数据类型（整型、字符型、浮点型、数组）、运算符（算术、关系、逻辑、位运算）、控制流语句（if-else、while、for）、函数定义和调用等核心语言特性，为后续各章的实现提供语言基础。

第三章为词法分析与语法分析，阐述词法分析和语法分析的实现方法。词法分析部分介绍 Flex 工具的使用、Token 的定义和识别、行列号的跟踪机制；语法分析部分详细说明文法设计、Bison 工具的配置、抽象语法树（AST）的构建方法，以及如何处理运算符优先级和结合性问题。

第四章为语义分析，详细介绍语义分析模块的设计与实现。包括符号表的数据结构设计、多层作用域管理、类型检查机制、22 种语义错误的检测方法。特别强调了函数声明与定义的一致性检查、变量作用域的正确处理、控制流语句的合法性验证等关键技术问题。

第五章为中间代码生成，阐述中间代码生成的原理和实现。介绍三地址码的设计、30 余种中间指令的定义、表达式翻译方法、控制流语句的代码生成策略。重点讨论短路求值的实现、函数调用的中间表示、数组访问的偏移量计算等技术细节。

第六章为目标代码生成，详细说明 MIPS 汇编代码的生成过程。包括 MIPS 指令集的使用、寄存器分配策略、栈帧布局设计、函数调用约定的实现。重点介绍激活记录的管理、递归函数的支持、数组访问的代码生成等关键技术。

第七章为系统测试，全面展示编译器的测试与验证。通过三个典型测试用例（`test.c`、`error.c`、`array.c`），完整演示从源代码到 MIPS 汇编代码再到 MARS 运行结果的全过程。重点验证语义错误检测的准确性、多维数组的处理能力、以及各编译阶段输出的正确性。

第八章为总结与展望，总结项目实现的技术成果和创新点，深入分析实现过程中遇到的技术难点及其解决方案，指出编译器存在的不足之处，并提出未来的改进方向（代码优化、浮点数支持、工具链集成等）。最后分享编译器开发的心得体会和技术展望。

2 语言定义

编译课程的实验，第一个需要完成的工作，就是要定义好待实现其编译器的语言，用上下文无关文法定义该语言，并给该语言起一个有意义的名称。后续的工作就是完成该语言的编译器。以下我们定义了 Mini-C 语言。

2.1 语言概述

Mini-C 是一种为编译原理教学设计的简化版 C 语言。该语言保留了 C 语言的核心特性，同时去除了复杂的预处理指令、指针操作、结构体等高级特性，使得编译器的实现更加聚焦于编译原理的核心概念。Mini-C 语言支持基本的数据类型、算术与逻辑运算、控制流语句、函数定义与调用以及多维数组等功能，足以编写具有一定复杂度的程序，如递归算法、排序算法等。

2.2 数据类型

Mini-C 支持三种基本数据类型：

表格 2-1

类型	关键字	说明	占用空间
字符型	char	存储单个字符，支持转义字符	4 字节
整型	int	存储整数	4 字节
浮点型	float	存储浮点数	4 字节

字符常量支持以下转义序列：`\n`（换行）、`\r`（回车）、`\t`（制表符）、`\0`（空字符）、`\'`（单引号）、`\\`（反斜杠）。

2.3 变量与数组

2.3.1 变量声明

Mini-C 支持全局变量和局部变量的声明，变量声明时可以带有初始化表达式：

```
1. int a;           // 简单变量声明
2. int a, b, c;      // 多变量声明
3. int x = 10;       // 带初始化的变量声明
4. float pi = 3.14;  // 浮点变量初始化
```

2.3.2 多维数组

Mini-C 支持全局变量和局部变量的声明，变量声明时可以带有初始化表达式：

```
1. int arr[10];      // 一维数组
2. int matrix[3][4]; // 二维数组
3. int cube[2][3][4]; // 三维数组
```

数组元素通过下标访问，下标从 0 开始：

```
1. arr[0] = 1;
2. matrix[i][j] = i * 4 + j;
```

2.4 运算符

2.4.1 算术运算符

Mini-C 支持全局变量和局部变量的声明，变量声明时可以带有初始化表达式：

表格 2-2

运算符	说明	示例
<code>+</code>	加法 / 正号	<code>a + b</code> 、 <code>+a</code>
<code>-</code>	减法 / 负号	<code>a - b</code> 、 <code>-a</code>
<code>*</code>	乘法	<code>a * b</code>
<code>/</code>	除法	<code>a / b</code>
<code>%</code>	取模	<code>a % b</code>

2.4.2 关系运算符

表格 2-3

运算符	说明	示例
>	大于	a > b
>=	大于等于	a >= b
<	小于	a < b
<=	小于等于	a <= b
==	等于	a == b
!=	不等于	a != b

2.4.3 逻辑运算符

表格 2-4

运算符	说明	示例
&&	逻辑与	a && b
	逻辑或	a b
!	逻辑非	!a

2.4.4 自增自减运算符

表格 2-5

运算符	说明	示例
++	前置自增	++a

<code>++</code>	后置自增	<code>a++</code>
<code>--</code>	前置自减	<code>--a</code>
<code>--</code>	后置自减	<code>a--</code>

2.4.5 赋值运算符

表格 2-6

运算符	说明	等价形式
<code>=</code>	简单赋值	<code>a = b</code>
<code>+=</code>	加法赋值	<code>a = a + b</code>
<code>-=</code>	减法赋值	<code>a = a - b</code>
<code>*=</code>	乘法赋值	<code>a = a * b</code>
<code>/=</code>	除法赋值	<code>a = a / b</code>
<code>%=</code>	取模赋值	<code>a = a % b</code>

2.5 控制语句

2.5.1 条件语句

```
1. // if 语句
2. if (condition)
3.     statement;
4.
5. // if-else 语句
6. if (condition)
7.     statement1;
8. else
9.     statement2;
```

2.5.2 循环语句

```
1. // while 循环
2. while (condition)
3.     statement;
4.
5. // for 循环
6. for (init; condition; update)
7.     statement;
```

2.5.3 跳转语句

```
1. break;      // 跳出当前循环
2. continue;   // 跳过本次循环，进入下一次迭代
3. return;     // 无返回值返回
4. return exp; // 带返回值返回
```

2.6 函数

Mini-C 支持函数的声明和定义，支持递归调用，其中内置两个系统函数，`read()`从标准输入读取一个整数，`write(exp)`将表达式的值输出到标准输出：

```
1. // 函数定义
2. int max(int a, int b) {
3.     if (a > b)
4.         return a;
5.     else
6.         return b;
7. }
8.
9. // 递归函数
10. int fibo(int n) {
11.     if (n == 1 || n == 2)
12.         return 1;
13.     return fibo(n - 1) + fibo(n - 2);
14. }
15.
16. // 主函数
17. int main() {
18.     int result;
19.     result = fibo(10);
```

```

20.     write(result);
21.     return 0;
22. }

```

2.7 注释

Mini-C 支持行注释和块注释两种注释形式：

```

1.  // 这是单行注释
2.
3.  /* 这是
4.     多行注释 */

```

2.8 文法描述

以下是 Mini-C 语言的形式化文法定义（使用 BNF 描述）：

program $\rightarrow$ ExtDefList

ExtDefList $\rightarrow$ ExtDef ExtDefList $\mid \epsilon$

ExtDef $\rightarrow$ Specifier ExtDecList SEMI
 $\mid$ Specifier ID LP ParamList RP CompSt
 $\mid$ Specifier ID LP ParamList RP SEMI

Specifier $\rightarrow$ TYPE

ExtDecList $\rightarrow$ VarDec $\mid$ VarDec COMMA ExtDecList

VarDec $\rightarrow$ ID $\mid$ VarDec LB INT RB

ParamList $\rightarrow$ ParamDec $\mid$ ParamList COMMA ParamDec $\mid \epsilon$

ParamDec $\rightarrow$ Specifier ID

CompSt $\rightarrow$ LC DefList StmList RC

DefList $\rightarrow$ Def DefList | ϵ

Def $\rightarrow$ Specifier DecList SEMI

DecList $\rightarrow$ Dec | Dec COMMA DecList

Dec $\rightarrow$ VarDec | VarDec ASSIGN Exp

StmList $\rightarrow$ Stmt StmList | ϵ

Stmt $\rightarrow$ Exp SEMI
| CompSt
| RETURN Exp SEMI
| RETURN SEMI
| IF LP Exp RP Stmt
| IF LP Exp RP Stmt ELSE Stmt
| WHILE LP Exp RP Stmt
| FOR LP Exp SEMI Exp SEMI Exp RP Stmt
| BREAK SEMI
| CONTINUE SEMI

Exp $\rightarrow$ Exp ASSIGN Exp
| Exp PLUS Exp | Exp MINUS Exp
| Exp STAR Exp | Exp DIV Exp | Exp MOD Exp
| Exp AND Exp | Exp OR Exp | NOT Exp
| Exp GT Exp | Exp GE Exp | Exp LT Exp
| Exp LE Exp | Exp EQ Exp | Exp NE Exp
| DPLUS Exp | DMINUS Exp | Exp DPLUS | Exp DMINUS

| LP Exp RP
 | MINUS Exp | PLUS Exp
 | ID LP Args RP
 | ID | ID SubList
 | INT | FLOAT | CHAR

Args $\rightarrow$ Exp | Args COMMA Exp | ϵ

SubList $\rightarrow$ LB Exp RB | SubList LB Exp RB

3 词法分析与语法分析

3.1 词法分析器设计

词法分析器是编译器的第一个阶段，负责将源程序的字符流转换为有意义的词法单元（Token）序列。本编译器使用 Flex 工具生成词法分析器。

3.1.1 词法分析器原理

词法分析器的核心任务是识别源代码中的各种词法单元，包括关键字、标识符、常量、运算符和分隔符等。Flex 是一个基于正则表达式的词法分析器生成工具，它采用有限自动机理论，通过最长匹配原则和优先匹配原则来处理模式冲突。

词法分析的核心任务是：

1. 识别词法单元：将字符序列划分为关键字、标识符、常量、运算符、分隔符等
2. 过滤无关信息：去除空白字符和注释
3. 记录位置信息：跟踪每个词法单元的行号和列号，便于错误定位
4. 返回词法单元：向语法分析器提供 Token 及其属性值

Flex 基于正则表达式进行模式匹配，采用最长匹配原则和优先匹配原则处理冲突。

3.1.2 词法单元分类

Mini-C 语言的词法单元主要分为以下几类：

1. **关键字**：Mini-C 共有 10 个关键字，包括类型关键字（int、float、char）、控制流关键字（if、else、while、for、return）和跳转关键字（break、continue）。这些关键字在词法分析器中需要优先于标识符进行匹配。
2. **运算符**：支持算术运算符（+、-、*、/、%）、关系运算符（>、>=、<、<=、==、!=）、逻辑运算符（&&、||、!）、赋值运算符（=、+=、-=、*=、/=、%=）以及自增自减运算符（++、--）。需要注意的是，多字符运算符（如>=、++）必须优先于单字符运算符匹配。

3. **标识符**：以字母或下划线开头，后跟任意数量的字母、数字或下划线。标识符的正则表达式定义为：`[A-Za-z_][A-Za-z0-9_]*`。
4. **常量**：支持整型常量、浮点常量和字符常量。整型常量支持十进制表示（0或非零开头的数字串），浮点常量支持小数和科学计数法，字符常量支持常见的转义字符（`\n`、`\t`、`\r`、`\0`、`\'`、`\\`）。
5. **分隔符**：包括分号、逗号、括号（小括号、中括号、花括号）等。
6. **注释**：支持单行注释（`//`）和多行注释（`/* */`），在词法分析阶段被直接过滤。

表格 3-1

关键字	Token 名称	说明
<code>int</code>	TYPE	整型类型
<code>float</code>	TYPE	浮点类型
<code>char</code>	TYPE	字符类型
<code>if</code>	IF	条件语句
<code>else</code>	ELSE	条件分支
<code>while</code>	WHILE	while 循环
<code>for</code>	FOR	for 循环
<code>break</code>	BREAK	跳出循环
<code>continue</code>	CONTINUE	继续循环
<code>return</code>	RETURN	返回语句

表格 3-2

运算符	Token 名称	运算符	Token 名称
<code>+</code>	PLUS	<code>+=</code>	PLUSASSIGN
<code>-</code>	MINUS	<code>-=</code>	MINUSASSIGN

*	STAR	*=	STARASSIGN
/	DIV	/=	DIVASSIGN
%	MOD	%=	MODASSIGN
++	DPLUS	--	DMINUS
>	GT	>=	GE
<	LT	<=	LE
==	EQ	!=	NE
&&	AND	,	
!	NOT	=	ASSIGN

表格 3-3

分隔符	Token 名称	说明
;	SEMI	语句结束
,	COMMA	逗号分隔
(	LP	左小括号
)	RP	右小括号
{	LC	左花括号
}	RC	右花括号
[	LB	左中括号
]	RB	右中括号

3.1.3 正则表达式定义

词法分析器使用以下正则表达式定义各类词法单元：

```
1. /* 标识符：以字母或下划线开头，后接字母、数字或下划线 */
2. id      [A-Za-z_][A-Za-z0-9_]*
3.
4. /* 整型常量：0 或 非零数字开头的数字串 */
5. intconst 0|([1-9][0-9]*)
6.
7. /* 浮点常量：支持小数形式和科学计数法 */
8. floatconst (([0-9]+\.[0-9]*)|([0-9]*\.[0-9]+))([eE][-+]?[0-9]+)?
9.
10. /* 字符常量：单引号包围，支持转义字符 */
11. charconst '([^'\\]|\\[nrt'\0])'
12.
13. /* 单行注释：// 到行尾 */
14. Oneline_comment "//"^[^n]*
15.
16. /* 多行注释：/* 到 */ 之间的内容 */
17. Multiline_comment "/*"([^\*]|\\*+[^\/])*\*+ "/"
```

3.1.4 关键技术

1. 位置跟踪机制

通过 `YY_USER_ACTION` 宏在每次匹配动作前自动更新位置信息：

- `yylloc.first_line / yylloc.last_line`：Token 的起止行号
- `yylloc.first_column / yylloc.last_column`：Token 的起止列号

这些信息用于后续的错误报告，能够精确定位错误位置。

2. 模式匹配优先级

Flex 的匹配规则遵循两个原则：（1）最长匹配原则——当多个模式都能匹配时，选择匹配字符串最长的规则；（2）优先匹配原则——当匹配长度相同时，选择先定义的规则。基于这些原则，关键字规则必须放在标识符规则之前，多字符运算符必须放在单字符运算符之前。

3. 字符常量处理

字符常量的处理需要特别注意转义字符的识别。例如对于输入 `'\n'`，需要将其转换为换行符的 ASCII 码值。

3. 错误处理

对于无法识别的字符，词法分析器会输出错误信息并记录错误数量，但不中断编译过程，以便发现更多潜在错误。

3.2 语法分析器设计

语法分析器是编译器的核心组件，负责根据文法规则分析 Token 序列的语法结构，并构建抽象语法树(AST)。本编译器使用 Bison 工具自动生成基于 LALR(1) 算法的语法分析器。

3.2.1 LALR(1)分析原理

本编译器采用 LALR(1) 自底向上分析方法：

- L：从左向右扫描输入
- A：最左归约（等价于最右推导的逆过程）
- LR：使用 LR 分析表
- (1)：向前看一个符号

Bison 根据文法规则自动构造 LALR(1)分析表，在运行时通过"移进-归约"操作完成语法分析。当遇到冲突时，可以通过声明运算符的优先级和结合性来解决。

3.2.2 文法结构设计

Mini-C 的文法设计采用层次化结构，主要包括以下几个层次：

1. 程序结构层：程序由外部定义列表组成，每个外部定义可以是全局变量声明或函数定义/声明。程序的起始符号为 `program`，其产生式为：

```
program: ExtDefList ExtDefList: /* 空 */ | ExtDef ExtDefList
```

2. 声明层：处理变量和函数的声明。变量声明支持简单变量和多维数组，函数声明包含返回类型、函数名、参数列表和函数体。例如变量声明的产生式：

VarDec: ID | VarDec LB INT RB /* 数组声明 */

3. 语句层：包括表达式语句、复合语句、条件语句、循环语句、跳转语句等。每种语句都对应特定的 AST 节点类型。

4. 表达式层：表达式是文法中最复杂的部分，涉及各种运算符的优先级和结合性。表达式包括赋值表达式、算术表达式、逻辑表达式、关系表达式、函数调用、数组访问等。

3.2.3 运算符优先级与结合性

表达式的正确解析依赖于运算符优先级和结合性的设置。Bison 通过 %left（左结合）、%right（右结合）和 %nonassoc（不可结合）声明来处理。优先级按照声明顺序从低到高排列：

1. %left COMMA	/* 逗号运算符，最低优先级 */
2. %right ASSIGN ...	/* 赋值运算符，右结合 */
3. %left OR	/* 逻辑或 */
4. %left AND	/* 逻辑与 */
5. %left NE EQ	/* 相等性判断 */
6. %left LT LE GT GE	/* 关系运算符 */
7. %left PLUS MINUS	/* 加减运算 */
8. %left STAR DIV MOD	/* 乘除取模 */
9. %right UMINUS NOT UPLUS	/* 一元运算符 */
10. %right DPLUS DMINUS	/* 自增自减 */
11. %left ARRPRO	/* 数组下标，最高优先级 */

3.2.4 语义动作与 AST 构建

每个文法产生式都可以附带一个语义动作，在归约时执行。语义动作的主要任务是创建相应的 AST 节点并设置其属性。例如，赋值表达式的语义动作：

```
1. Exp: Exp ASSIGN Exp {
2.     AssignAST *t = new AssignAST();
3.     t->Op = ASSIGN;
4.     t->LeftValExp = $1;
5.     t->RightValExp = $3;
6.     $$ = t;
7.     SavePosition;
8. }
```

其中 \$1、\$3 分别表示产生式右部的第 1 和第 3 个符号的语义值，\$\$ 表示归

约后的非终结符的语义值，`SavePosition` 是一个宏，用于记录 AST 节点的源代码位置。

3.2.5 悬空 else 问题的解决

悬空 else (Dangling Else) 是语法分析中的经典二义性问题。对于以下代码：

```
1. if (a > 0)
2.     if (b > 0)
3.         c = 1;
4.     else
5.         c = 2;
```

`else` 可以与第一个或第二个 `if` 匹配。按照 C 语言惯例，`else` 应与最近的未匹配 `if` 配对。Bison 的解决方案是通过设置 `ELSE` 终端符号的优先级高于无 `else` 的 `if` 语句(使用虚拟的 `%nonassoc LOWER_THEN_ELSE` 占位符)来解决移进-归约冲突。当遇到移进-归约冲突时，由于 `ELSE` 优先级高于 `LOWER_THEN_ELSE`，分析器会选择移进 `ELSE`，从而实现就近匹配：

```
1. %nonassoc LOWER_THEN_ELSE /* 虚拟优先级，低于 ELSE */
2. %nonassoc ELSE           /* ELSE 优先级较高 */
3.
4. Stmt: IF LP Exp RP Stmt %prec LOWER_THEN_ELSE /* 无 else 的 if */
5.      | IF LP Exp RP Stmt ELSE Stmt           /* 有 else 的 if */
```

3.3 抽象语法树 (AST) 设计

抽象语法树是源程序的树形表示，是语法分析的输出和后续语义分析的输入。

3.3.1 AST 节点类型体系

本编译器的 AST 采用面向对象的设计，所有节点类都继承自基类 `AST`。整个类型体系按照语言结构分为以下几大类：

程序和定义节点：

- `ProgAST`：程序根节点，包含外部定义列表
- `ExtDefAST`：外部定义基类，派生出 `ExtVarDefAST`（全局变量定义）和

FuncDefAST（函数定义）

类型和声明节点：

- TypeAST：类型基类，目前仅有 BasicTypeAST（基本类型）
- VarDecAST：变量声明节点，支持简单变量和多维数组
- ParamAST：函数形参节点

语句节点：

- StmAST：语句基类，派生出多种具体语句类型
- CompStmAST：复合语句（代码块）
- IfStmAST 和 IfElseStmAST：条件语句
- WhileStmAST 和 ForStmAST：循环语句
- ReturnStmAST、BreakStmAST、ContinueStmAST：跳转语句
- ExprStmAST：表达式语句

表达式节点：

- ExpAST：表达式基类
- ConstAST：常量表达式
- VarAST：变量引用（支持数组下标访问）
- BinaryExprAST：二元运算表达式
- UnaryExprAST：一元运算表达式
- AssignAST：赋值表达式
- FuncCallAST：函数调用表达式

3.3.2 AST 节点设计要点

每个 AST 节点都包含以下核心信息：

1. **位置信息**：Line 和 Column 成员记录了该语法结构在源代码中的位置，用于错误报告和调试。
2. **语义属性**：不同类型的节点包含不同的语义信息。例如 VarAST 包含变量名和数组下标列表，BinaryExprAST 包含运算符类型和左右操作数。
3. **符号表引用**：某些节点（如 VarAST、FuncCallAST）包含指向符号表项的指针，用于语义分析和中间代码生成阶段快速访问符号信息。

4. 虚函数接口：基类定义了 `DisplayAST()`（AST 可视化）、`Semantics()`（语义检查）和 `GenIR()`（中间代码生成）等虚函数，各派生类根据自身特点实现这些接口。

3.3.3 AST 可视化输出

为了便于调试和理解编译器的工作过程，实现了 AST 的格式化输出功能。输出采用缩进式层次结构，清晰展示程序的语法树结构。例如，对于一个简单的函数定义：

```
1. int max(int a, int b) {  
2.     if (a > b)  
3.         return a;  
4.     else  
5.         return b;  
6. }
```

其 AST 输出格式为：

```
1. 函数定义：  
2.     返回类型：int  
3.     函数名：max  
4.     形参列表  
5.         int a  
6.         int b  
7.     函数体：  
8.         复合语句：  
9.             语句列表：  
10.                if-else 语句：  
11.                    条件：  
12.                        >  
13.                        a  
14.                        b  
15.                    满足条件执行：  
16.                        返回表达式：  
17.                            a  
18.                    否则执行：  
19.                        返回表达式：  
20.                            b
```

3.4 错误处理机制

3.4.1 错误检测与报告

编译器实现了完善的错误检测和报告机制。词法错误和语法错误都会被精确记录，包括错误位置（行号、列号）和错误描述。

词法错误通过模式 `.`（匹配任意字符）捕获，对于无法识别的字符，输出类似以下的错误信息：“在第 15 行出现不可识别的符号 '@' ”，语法错误通过 Bison 的 `yyerror` 函数报告，该函数接收词法分析器提供的位置信息：

```
1. void yyerror(const char* fmt, ...) {  
2.     Errors::ErrorAdd(yylloc.first_line, yylloc.first_column, fmt);  
3. }
```

所有错误被收集到 `Errors::Errs` 向量中，在编译结束时统一显示，避免错误信息被中间输出淹没。

3.4.2 错误恢复机制

为了在单个错误后继续分析以发现更多错误，语法分析器采用了 `panic-mode` 错误恢复机制。当遇到语法错误时，分析器会丢弃输入符号直到遇到同步符号（通常是分号、右花括号等），然后恢复正常分析。

在文法中通过 `error` 符号实现错误恢复：

```
1. Stmt: error SEMI { $$ = NULL; } /* 遇到错误时跳过直到分号 */
```

这种机制虽然简单，但能有效避免单个语法错误导致大量连锁错误报告，提高了错误信息的可读性。

3.5 编译流程总结

词法分析和语法分析构成了编译器前端的主体部分，其工作流程可以概括为：

1. 词法分析阶段：Flex 生成的词法分析器读取源文件字符流，根据正则表达式规则识别 Token，过滤注释和空白，记录位置信息，将 Token 序列传递给语法分析器。

2. 语法分析阶段：Bison 生成的语法分析器接收 Token 序列，根据语法规则进行归约操作，在归约过程中执行语义动作构建 AST 节点，最终生成完整的抽象语法树。

3. 错误处理：两个阶段都实现了错误检测和报告机制，词法错误和语法错误被收集后统一显示，提供精确的位置信息。

4. AST 输出：在 DISPLAY_AST 宏开启时，会输出格式化的抽象语法树，便于调试和验证语法的正确性。

通过 Flex 和 Bison 工具的配合使用，编译器前端实现了从源代码到抽象语法的自动化转换，为后续的语义分析和中间代码生成奠定基础。

4 符号表管理与语义分析

4.1 概述

语义分析是编译器前端的最后一个阶段，它在语法分析构建的抽象语法树基础上，进行静态语义检查并收集类型信息。语义分析的核心任务是确保程序在语义上是正确的，即符合语言的类型系统和作用域规则。本编译器的语义分析模块基于AST 进行遍历，通过符号表管理作用域信息，完成类型检查、作用域解析和各类语义错误检测。其主要职责包括：

- 1 **符号表管理**：维护一个基于栈结构的符号表，支持嵌套作用域的符号查找与管理
- 2 **类型检查**：验证表达式、赋值、函数调用等操作中的类型一致性
- 3 **作用域解析**：处理变量和函数的声明与引用，确保标识符在使用前已正确定义
- 4 **属性计算**：计算变量的存储偏移量、数组的维度信息、函数的活动记录大小等
- 5 **错误检测**：识别并报告各类静态语义错误，提供精确的错误位置信息

4.2 符号表设计

4.2.1 符号表数据结构

符号表是编译器中用于存储标识符信息的核心数据结构。本编译器采用**栈式符号表**结构，以支持 Mini-C 语言的块级作用域特性。

符号基类 (Symbol)：所有符号的抽象基类，包含三个基本属性：符号名称 (Name)、符号类型 (Type, 可为 T_CHAR、T_INT 或 T_FLOAT)、符号种类 (Kind, 用单字符标识：'V'表示普通变量、'F'表示函数、'P'表示参数、'A'表示数组)。

变量符号 (VarSymbol)：继承自 Symbol，用于表示变量、参数和数组。除基类属性外，还包含：别名 (Alias, 用于中间代码生成时区分同名变量)、偏移量 (Offset, 变量在活动记录中的位置)、全局标志 (isGlobal, 标识是否为全局变量)、维度信息 (Dims, 数组各维的大小)。

函数符号 (FuncSymbol): 继承自 Symbol, 用于表示函数定义和声明。包含: 活动记录大小 (ARSize)、形参个数 (ParamNum)、参数符号表指针 (ParamPtr)、声明/定义标志 (Declaration, 区分函数声明和定义)、形参列表 (Params)。

单一作用域符号表 (SymbolsInAScope): 表示一个作用域内的所有符号, 内部使用向量存储该作用域的符号指针列表。

符号表栈 (SymbolStackDef): 整个符号表的顶层结构, 使用向量模拟栈, 存储多个作用域符号表的指针。栈底存放全局符号 (全局变量和函数), 每进入一个新的复合语句块就压入一个新的作用域符号表, 离开时弹出。

4.2.2 符号表管理功能

当前作用域查找 (LocateNameCurrent): 仅在栈顶的当前作用域中查找指定名称的符号。该函数用于检测同一作用域内的重复定义, 返回找到的符号指针或 NULL。

全局作用域查找 (LocateNameGlobal): 从栈顶向栈底遍历所有作用域, 查找指定名称的符号。这种由内向外的查找顺序确保了内层作用域的符号会遮蔽外层同名符号, 实现了正确的作用域规则。

别名生成 (NewAlias): 为每个变量生成唯一的别名字符串 (如 v0、v1、v2...), 用于在中间代码中区分不同作用域的同名变量, 解决变量名的二义性问题。

作用域进入与退出: 进入新的复合语句时, 创建新的 SymbolsInAScope 对象并压入符号表栈; 离开复合语句时, 将该作用域从栈中弹出。这种机制自然地支持了嵌套作用域的管理。

4.2.3 符号表动态变化过程

编译器提供了符号表动态显示功能 (通过 DISPLAY_SYMBOL_TABLE 宏控制), 可以在编译过程中实时观察符号表的变化。以下是一个示例程序编译时符号表的变化过程:

源程序:

```
1. int g;  
2. int foo(int x) {  
3.     int y;  
4.     return x + y;
```

```
5. }
```

符号表变化过程：

```
1. [符号表] 添加全局变量: int g (别名:v0, 偏移:0)
2. [符号表] 定义函数: int foo() - 参数个数:1
3. [符号表] 进入函数 foo 的作用域 (层次 1)
4. [符号表] 添加函数参数: int x (别名:v1, 偏移:12)
5. [符号表] 添加局部变量: int y (别名:v2, 偏移:16)
6.
7. >>> 当前复合语句符号表状态 <<<
8. >>> ===== 符号表层次结构 =====
9. >>> [层次 0] 全局作用域
10. >>> 变量: int g (别名:v0, 偏移:0, 全局)
11. >>> 函数: int foo() 参数:1 AR 大小:20
12. >>> [层次 1] 局部作用域
13. >>> 参数: int x (别名:v1, 偏移:12)
14. >>> 变量: int y (别名:v2, 偏移:16)
15. >>> =====
16. >>>
17. >>
18.
19. [符号表] 离开复合语句作用域 (层次 1)
20. [符号表] 函数 foo AR 大小: 20
```

这种分层显示清晰地展示了符号表的栈结构：层次 0 为全局作用域，包含全局变量和函数定义；层次 1 及以上为局部作用域，包含函数参数和局部变量。每个符号都显示了其类型、别名和偏移量等关键属性。

4.3 属性计算

4.3.1 存储偏移量计算

编译器为每个变量计算其在活动记录（Activation Record, AR）中的偏移量。偏移量的计算遵循以下规则：

全局变量：全局变量的偏移量从 0 开始，依次累加。每个变量占用的空间由其类型决定（本编译器中 char、int、float 均占用 4 字节）。数组的空间为元素大小乘以各维大小的乘积。

局部变量和参数：函数的活动记录从偏移量 12 开始分配（前 12 字节预留给返

回地址、动态链和静态链)。参数和局部变量依次分配，每个占用 4 字节。编译器会记录函数所需的最大活动记录大小 (ARSize)，用于运行时栈空间分配。

4.3.2 类型推导

表达式的类型通过属性计算自底向上传递。二元运算表达式的类型推导规则为：

- 若任一操作数为 float 类型，结果为 float
- 否则若任一操作数为 int 类型，结果为 int
- 若两操作数均为 char 类型，结果为 char

一元运算表达式的类型与其操作数类型相同。函数调用表达式的类型为函数的返回类型。

4.3.3 数组维度信息

对于数组声明，编译器会提取并保存各维的大小信息。例如对于声明 int arr[3][4][5]，会在符号表中记录维度向量 {3, 4, 5}。这些信息在数组访问时用于：

- 验证下标表达式的维度数是否正确
- 中间代码生成时计算数组元素的地址偏移

4.4 静态语义检查

本编译器实现了全面的静态语义错误检测，能够识别 20 种以上的语义错误。以下按类别详细说明：

4.4.1 变量相关错误

表格 4-1

错误编号	错误类型	说明	示例
错误 1	变量重复定义	在同一作用域内重复声明同名变量	<code>int a; int a;</code>
错误 2	使用未定义变	引用未声明的标识符	<code>x = 10;</code>

	量		(x 未声明)
错误 3	对非数组使用下标	对普通变量使用数组下标访问	<code>int a; a[0] = 1;</code>
错误 4	数组维度不匹配	访问数组时下标个数与声明维度不符	<code>int a[3][4]; a[0] = 1;</code>
错误 5	数组下标类型错误	数组下标不是整型表达式	<code>int a[10]; a[1.5] = 0;</code>
错误 6	数组初始化错误	数组声明时使用表达式初始化	<code>int a[10] = 0;</code>

4.4.2 函数相关错误

表格 4-2

错误编号	错误类型	说明	示例
错误 7	函数重复定义	重复定义同名函数	两次定义 <code>int foo() {...}</code>
错误 8	调用未定义函数	调用不存在的函数	<code>bar();</code> (bar 未定义)
错误 9	非函数标识符调用	对变量使用函数调用语法或对函数名使用非调用表达式	<code>int a; a();</code> 或 <code>int foo(); foo = 1;</code>
错误 10	实参数量不匹配	函数调用时实参个数与形参不符	<code>int foo(int a);</code> <code>foo(1,2);</code>

错 误 11	实参类型不匹配	实参类型与形参类型不一致	<code>int foo(int a);</code> <code>foo(1.5);</code>
错 误 12	声明与定义返回类型不一致	函数声明和定义的返回类型不同	声明 <code>int foo();</code> 定义 <code>float foo(){...}</code>
错 误 13	声明与定义参数数量不一致	声明和定义参数个数不同	声明 <code>int foo(int a);</code> 定义 <code>int foo(int a, int b){...}</code>
错 误 14	声明与定义参数类型不一致	声明和定义参数类型不匹配	声明 <code>int foo(int a);</code> 定义 <code>int foo(float a){...}</code>
错 误 15	参数名重复	函数参数列表中有同名参数	<code>int foo(int a, int a)</code>
错 误 18	函数缺少返回语句	函数没有 <code>return</code> 语句	<code>int foo() { int a; }</code>

4.4.3 语句相关错误

表格 4-3

错 误 编 号	错误类型	说明	示例
------------	------	----	----

错误 16	对非左值赋值或自增自减	赋值表达式左侧不是可修改的左值，或对非左值执行++/-	<code>1 = a;</code> 、 <code>(a+b)++;</code>
错误 17	返回类型不匹配	<code>return</code> 语句的值类型与函数返回类型不符	<code>int foo() { return 1.5; }</code>
错误 19	<code>break/continue</code> 不在循环内	<code>break</code> 或 <code>continue</code> 语句出现在循环外	在 <code>if</code> 语句中直接使用 <code>break</code>

4.4.4 程序结构错误

表格 4-4

错误类型	说明
缺少 <code>main</code> 函数	程序中没有定义 <code>main</code> 函数
<code>main</code> 函数返回类型错误	<code>main</code> 函数的返回类型不是 <code>int</code>
<code>main</code> 函数带参数	<code>main</code> 函数定义了参数（Mini-C 不支持命令行参数）
函数只有声明无定义	函数被声明并调用，但没有提供定义

说明：编号 1-19 对应代码中的具体错误标识，程序结构错误在全局检查阶段执行。共计 19 种编号错误 + 3 种程序结构错误（缺少 `main`、`main` 返回类型错误、`main` 带参数实际算 `main` 函数相关的检查，函数只声明未定义算 1 种）= **22 种语义错误检查**。

4.4.5 左值判断

左值（L-value）是指可以出现在赋值表达式左侧的表达式，即可被修改的存储位置。编译器通过递归检查来判断表达式是否为左值：

- 变量引用（VarAST）是左值

- 数组元素访问是左值（如 `arr[i]`）
- 常量、字面量不是左值
- 二元运算结果不是左值
- 函数调用结果不是左值

自增自减运算符（`++`、`--`）和赋值运算符要求其操作对象必须是左值，否则报告语义错误。

4.5 语义分析流程

语义分析采用深度优先遍历 AST 的方式进行，通过每个 AST 节点类的 `Semantics` 虚函数实现多态调用。整体流程如下：

- 1 **初始化阶段**：创建全局作用域符号表，预定义系统函数 `read` 和 `write`
- 2 **全局定义处理**：遍历所有外部定义（`ExtDef`），处理全局变量声明和函数定义/声明
 - 全局变量：检查重定义，计算偏移量，加入全局符号表
 - 函数：检查重定义，处理参数列表，创建函数作用域
- 3 **函数体分析**：对每个函数体进行递归分析
 - 进入函数作用域，处理参数声明
 - 遍历局部声明和语句列表
 - 对复合语句创建新的嵌套作用域
 - 检查 `return` 语句的类型匹配
- 4 **表达式分析**：自底向上计算表达式类型
 - 变量引用：查找符号表，获取类型信息
 - 函数调用：验证函数存在性、参数匹配
 - 运算表达式：根据操作数类型推导结果类型
- 5 **收尾检查**：验证 `main` 函数存在且签名正确，检查所有声明的函数是否都有定义

4.6 错误处理与恢复

语义分析阶段采用容错处理策略，遇到错误时不立即终止，而是尽可能继续分析以发现更多潜在问题。

错误信息包含三个要素：行号、列号和错误描述。所有错误被收集到统一的错误列表中，在编译结束后按位置排序输出，便于开发者定位和修复问题。

对于某些错误(如使用未定义变量)，编译器会给表达式赋予默认类型(通常为 `int`)，使后续分析能够继续进行。这种策略可能导致级联错误，但实践中能够有效帮助开发者发现更多独立的语义问题。

5 中间代码生成

5.1 概述

中间代码是源程序的一种抽象表示形式，位于高级语言和目标机器代码之间。使用中间代码的主要优势在于：（1）使编译器结构模块化，前端生成中间代码，后端将中间代码转换为目标代码；（2）便于进行机器无关的优化；（3）支持多种目标平台的代码生成。

本编译器采用三地址码（Three-Address Code）作为中间表示形式。三地址码的每条指令最多包含三个操作数：两个源操作数和一个目标操作数，形如 $x := y \text{ op } z$ 。这种形式简洁而富有表达力，能够表示各种程序结构。

5.2 三地址码设计

5.2.1 操作数结构

中间代码的操作数（Opn）包含以下信息：

- **Name:** 变量别名或函数名。特殊值"`_CONST`"表示该操作数为常量
- **Type:** 操作数的数据类型（`T_CHAR`、`T_INT` 或 `T_FLOAT`）
- **Offset:** 变量在活动记录中的偏移量
- **isGlobal:** 标识是否为全局变量
- **常量值:** 使用联合体存储不同类型的常量值（`constCHAR`、`constINT`、`constFLOAT`）

5.2.2 四元式结构

每条中间代码指令表示为一个四元式（IRCode），包含四个字段：

Op: 操作码，标识指令类型

Opn1: 第一操作数

Opn2: 第二操作数

Result: 结果操作数

5.2.3 指令类型

本编译器支持以下几类中间代码指令：

1. 算术运算指令：

表格 5-1

指令	格式	含义
PLUS	Result := Opn1 + Opn2	加法运算
MINUS	Result := Opn1 - Opn2	减法运算
STAR	Result := Opn1 * Opn2	乘法运算
DIV	Result := Opn1 / Opn2	除法运算
MOD	Result := Opn1 % Opn2	取模运算
UMINUS	Result := - Opn1	取负
DPLUS	Result := ++Opn1	自增
DMINUS	Result := --Opn1	自减

2. 关系运算指令

表格 5-2

指令	格式	含义
GT/GE/LT/LE/EQ/NE	Result := Opn1 op Opn2	比较运算，结果为 0 或 1

3. 控制流指令

表格 5-3

指令	格式	含义
----	----	----

LABEL	LABEL Result:	定义标号
GOTO	GOTO Result	无条件跳转
JGT/JGE/JLT/JLE/JEQ/JNE	IF Opn1 op Opn2 GOTO Result	条件跳转

4. 数据传送指令

表格 5-4

指令	格式	含义
ASSIGN	Result := Opn1	赋值
ARRLOAD	Result := Opn1[Opn2]	数组元素读取
ARRASSIGN	Result[Opn1] := Opn2	数组元素赋值

5. 函数调用指令

表格 5-5

指令	格式	含义
FUNCTION	FUNCTION Result:	函数入口定义
PARAM	PARAM Result	形参声明
ARG	ARG Result	实参传递
CALL	Result := CALL Opn1	函数调用（有返回值）
RETURN	RETURN Result	函数返回
END	End Of Program	程序结束标记

5.3 临时变量与标号处理

5.3.1 临时变量生成

表达式求值过程中产生的中间结果需要存储在临时变量中。编译器使用 `NewTemp()` 函数生成唯一的临时变量名，命名格式为 "Temp_1"、"Temp_2" 等。临时变量在函数的活动记录中分配空间，其偏移量从局部变量区之后开始。编译器会跟踪表达式求值过程中临时变量占用的最大空间 (`MaxTempVarOffset`)，最终将其加入函数的活动记录大小计算中。

5.3.2 标号生成

控制流语句需要使用标号来标识跳转目标。`NewLabel()` 函数生成唯一的标号名，命名格式为 "Label_1"、"Label_2" 等。标号用于：if 语句的 then 分支入口、else 分支入口和出口，while 循环的条件判断入口、循环体入口和出口，for 循环的各个控制点和短路求值中的中间跳转点

5.4 表达式的中间代码生成

5.4.1 常量表达式

常量的处理相对简单：生成一个临时变量，将常量值赋给该临时变量，返回临时变量作为表达式的结果。常量在四元式中用特殊名称 "_CONST" 标识，实际值存储在操作数的联合体成员中。

5.4.2 变量引用

对于普通变量 (Kind 为 'V' 或 'P')，直接返回其在符号表中记录的别名和偏移量信息。

对于数组元素访问，需要计算元素的线性偏移量。对于多维数组 `arr[i][j][k]`，采用行主序计算偏移： $offset = ((i * dim2 + j) * dim3 + k) * element_size$ 。这个计算过程会生成一系列乘法和加法指令，最终通过 `ARRLOAD` 指令读取数组元素。••

5.4.3 二元运算表达式

二元运算的代码生成遵循以下步骤：

- 1 递归生成左操作数的代码，得到左操作数结果
- 2 递归生成右操作数的代码，得到右操作数结果
- 3 生成临时变量存储运算结果
- 4 将左右操作数的代码序列和运算指令连接起来

5.4.4 逻辑表达式与短路求值

对于逻辑运算（&&和||），编译器实现了**短路求值**（Short-circuit Evaluation）。这种求值方式在能确定结果时不计算后续表达式，既提高效率又符合C语言语义。短路求值通过条件跳转实现。对于 $A \&\& B$ ：先计算 A，若 A 为假则直接跳转到整个表达式为假的标号，否则继续计算 B。对于 $A \parallel B$ ：先计算 A，若 A 为真则直接跳转到整个表达式为真的标号，否则继续计算 B。

表达式的 GenIR 方法有两个重载版本：一个返回表达式的值（用于普通求值），另一个接收真假标号参数（用于条件跳转）。条件语句和循环语句使用后一种形式来直接生成控制流代码。

逻辑运算指令：除了短路求值，逻辑运算符 AND、OR、NOT 也可作为普通二元/一元运算符使用，生成布尔值结果（0 或 1）。这用于将逻辑表达式的值赋给变量或作为算术表达式的一部分：

- AND: $\text{Result} := \text{Opn1} \&\& \text{Opn2}$ （按位与，操作数先归一化为 0/1）
- OR: $\text{Result} := \text{Opn1} \parallel \text{Opn2}$ （按位或，操作数先归一化为 0/1）
- NOT: $\text{Result} := !\text{Opn1}$ （逻辑非）

5.4.5 赋值表达式

普通变量赋值生成一条 ASSIGN 指令。数组元素赋值需要先计算下标偏移，再使用 ARRASSIGN 指令将值存入数组。

复合赋值运算符（+=、-=等）在语法分析阶段被展开为普通的算术运算和赋值操作

5.5 语句的中间代码生成

5.5.1 条件语句

if 语句的代码结构：

1. 计算条件表达式（条件为真跳转到 LabelThen，为假跳转到 LabelEnd）
2. lThen:
3. then 分支代码
4. lEnd:

if-else 语句的代码结构：

1. 计算条件表达式（条件为真跳转到 LabelThen，为假跳转到 LabelElse）
2. LabelThen:
3. then 分支代码
4. GOTO LabelEnd
5. LabelElse:
6. else 分支代码
7. LabelEnd:

5.5.2 循环语句

while 语句的代码结构：

1. LoopCond:
2. 计算条件表达式（为真跳转到 LoopEntry，为假跳转到 LoopEnd）
3. LoopEntry:
4. 循环体代码
5. GOTO LoopCond
6. LoopEnd:

for 语句的代码结构：

1. 初始化表达式代码
2. LoopCond:
3. 计算条件表达式（为真跳转到 LoopEntry，为假跳转到 LoopEnd）
4. LoopEntry:
5. 循环体代码
6. LoopContinue:
7. 更新表达式代码
8. GOTO LoopCond
9. LoopEnd:

for 循环引入了额外的 LoopContinue 标号, 用于支持 continue 语句——continue 跳转到更新表达式而非条件判断。

5.5.3 跳转语句

break 语句: 生成一条 GOTO 指令, 跳转到当前循环的出口标号 (LoopEnd)。

continue 语句: 生成一条 GOTO 指令, 跳转到当前循环的继续点——while 循环跳转到条件判断 (LoopCond), for 循环跳转到更新表达式 (LoopContinue)。

return 语句: 如果有返回表达式, 先计算表达式的值, 然后生成 RETURN 指令。

5.6 函数的中间代码生成

函数的中间代码包含以下部分:

FUNCTION 指令: 标记函数入口, Result 域存储函数名, Offset 域存储活动记录大小。

PARAM 指令序列: 为每个形参生成一条 PARAM 指令。

函数体代码: 局部声明和语句的中间代码。

END 指令 (仅 main 函数): 标记程序结束。

函数调用的代码生成:

依次计算各实参表达式。

生成 ARG 指令序列传递实参。

生成 CALL 指令调用函数。

返回值存储在临时变量中。

5.7 中间代码示例

以下是一个简单程序及其对应的中间代码:

```
1. int main() {
```

```
2.   int a, b, c;
3.   a = 5;
4.   b = 3;
5.   c = a + b * 2;
6.   write(c);
7.   return 0;
8. }
```

对应的中间代码:

```
1. FUNCTION main:
2.   v0 := #5
3.   v1 := #3
4.   Temp_1 := #2
5.   Temp_2:= v1*Temp_1
6.   Temp_3:= v0+Temp_2
7.   v2 := Temp_3
8.   ARG v2
9.   Temp_4 := CALL write
10.  RETURN #0
11.  End Of Program
```

6 目标代码生成

6.1 概述

目标代码生成是编译器的最后阶段，将中间代码转换为特定目标机器的汇编代码或机器代码。本编译器选择 MIPS 汇编语言作为目标代码，原因包括：（1）MIPS 架构简洁规整，指令集易于理解；（2）有成熟的模拟器（如 MARS、SPIM）支持程序调试和运行；（3）MIPS 是 RISC 架构的代表，适合教学和实验。

目标代码生成的主要任务包括：

- 1 指令选择：为每条中间代码选择对应的 MIPS 汇编指令序列
- 2 寄存器分配：将变量和临时值映射到寄存器或内存
- 3 运行时环境管理：处理函数调用约定、栈帧管理、参数传递
- 4 地址计算：处理全局变量、局部变量和数组的地址

6.2 MIPS 体系结构与寄存器约定

6.2.1 寄存器使用约定

MIPS 架构提供 32 个通用寄存器，本编译器遵循标准的 MIPS 调用约定：

表格 6-1

寄存器	名称	用途
zero(zero (zero(0)	常量零	恒为 0，只读
v0-v0-v0-v1	返回值	函数返回值
a0-a0-a0-a3	参数	函数调用参数（本编译器\$a0 用于 write 函数）

t0-t0-t0- t9	临时寄存器	表达式求值的中间结果
sp(sp (sp(29)	栈指针	指向当前栈顶
ra(ra (ra(31)	返回地址	函数调用的返回地址
gp(gp (gp(28)	全局指针	指向全局数据区

6.2.2 内存分布

MIPS 程序的内存布局分为以下区域：

- 代码段（.text）：存放程序指令
- 数据段（.data）：存放全局变量和常量字符串
- 栈区：从高地址向低地址增长，存放函数的活动记录

6.3 运行时存储组织

6.3.1 活动记录结构

每个函数调用对应一个活动记录（Activation Record，也称栈帧），包含以下内容：

1. 高地址
2. +-----+
3. | 返回地址 (\$ra) | 偏移 0
4. +-----+
5. | 动态链 | 偏移 4（保留，未使用）
6. +-----+
7. | 静态链 | 偏移 8（保留，未使用）
8. +-----+
9. | 第一个参数 | 偏移 12
10. +-----+
11. | 第二个参数 | 偏移 16
12. +-----+

```

13. | ... |
14. +-----+
15. | 局部变量 |
16. +-----+
17. | 临时变量 |
18. +-----+ <- $sp (栈指针)
19. 低地址

```

前 12 字节为管理信息，参数和局部变量从偏移 12 开始分配。

6.3.2 全局变量访问

全局变量通过全局指针 `gp` 访问。本编译器在数据段为全局变量预留空间，全局变量的偏移量相对于 `gp` 访问。本编译器在数据段为全局变量预留空间，全局变量的偏移量相对于 `gp` 访问。本编译器在数据段为全局变量预留空间，全局变量的偏移量相对于 `gp` 计算。

局部变量和临时变量通过栈指针 `$sp` 访问，偏移量在语义分析阶段已经计算好。

6.4 内存访问辅助函数

为了简化代码生成，编译器实现了两组辅助函数：

LoadFromMem 函数：生成从内存加载数据到寄存器的指令。根据变量是全局还是局部，自动选择使用 `gp` 或 `gp` 或 `gp` 或 `sp` 作为基址寄存器。例如：`lw $t1, 12($sp)` 将栈偏移 12 处的值加载到 `$t1`。

StoreToMem 函数：生成将寄存器数据存入内存的指令。同样自动处理全局和局部变量的区别。例如：`sw $t1, 16($sp)` 将 `$t1` 的值存入栈偏移 16 处。

对于数组访问，提供了重载版本，可以先计算偏移量再访问。这些函数接受两个操作数参数，将第二个操作数作为索引，动态计算实际地址。

6.5 中间代码到 MIPS 指令的翻译

6.5.1 赋值指令

常量赋值：使用 `li` (load immediate) 指令将立即数加载到寄存器，然后存入目标位置。

1. 中间代码: `v0 := #5`
2. MIPS 代码:
3. `li $t1, 5`
4. `sw $t1, 12($sp)`

变量赋值：先加载源操作数到寄存器，再存入目标位置。

1. 中间代码: `v2 := v0`
2. MIPS 代码:
3. `lw $t1, 12($sp)`
4. `sw $t1, 20($sp)`

6.5.2 算术运算指令

二元运算遵循"加载-计算-存储"模式：

1. 中间代码: `Temp_2 := v0 + v1`
2. MIPS 代码:
3. `lw $t1, 12($sp)   # 加载 v0`
4. `lw $t2, 16($sp)   # 加载 v1`
5. `add $t3, $t1, $t2 # 执行加法`
6. `sw $t3, 24($sp)   # 存储结果`

除法和取模：MIPS 的 `div` 指令将商存入 LO 寄存器，余数存入 HI 寄存器。使用 `mflo` (move from LO) 获取商，`mfhi` (move from HI) 获取余数。

自增自减：使用 `addi` (add immediate) 指令实现。自增时加 1，自减时加-1。

6.5.3 关系运算与条件跳转

条件跳转指令直接翻译为 MIPS 的分支指令：

表格 6-2

中间代码	MIPS 指令
IF $x > y$ GOTO L	bgt t1,t1, t1, t2, L
IF $x \geq y$ GOTO L	bge t1,t1, t1, t2, L
IF $x < y$ GOTO L	blt t1,t1, t1, t2, L
IF $x \leq y$ GOTO L	ble t1,t1, t1, t2, L
IF $x == y$ GOTO L	beq t1,t1, t1, t2, L
IF $x != y$ GOTO L	bne t1,t1, t1, t2, L

6.5.4 逻辑运算

逻辑运算需要将操作数归一化为布尔值（0 或 1）。使用 `sne`（set if not equal）指令将非零值转为 1：

1. 中间代码：Temp := x && y
2. MIPS 代码：
3. lw \$t1, offset_x(\$sp)

```

4.    sne $t1, $t1, $zero    # 将 x 转为布尔值
5.    lw $t2, offset_y($sp)
6.    sne $t2, $t2, $zero    # 将 y 转为布尔值
7.    and $t3, $t1, $t2      # 按位与
8.    sw $t3, offset_result($sp)

```

逻辑非使用两次归一化：先将非零转为 1，再与 0 比较得到反转结果。

6.5.5 数组访问

数组元素读取（ARRLOAD）：计算偏移量后加载元素。

```

1. 中间代码: Result := arr[offset]
2. MIPS 代码:
3.    lw $t4, offset_index($sp)    # 加载索引
4.    add $sp, $sp, $t4             # 调整基址
5.    lw $t1, offset_arr($sp)      # 加载元素
6.    sub $sp, $sp, $t4             # 恢复基址
7.    sw $t1, offset_result($sp)   # 存储结果

```

数组元素赋值（ARRASSIGN）：类似读取，但最后一步是存储而非加载。

6.6 控制流指令翻译

标号（LABEL）：直接翻译为 MIPS 标号，格式为 LabelName:。

无条件跳转（GOTO）：翻译为 j LabelName 指令。

条件跳转：如 5.5.3 节所述，翻译为相应的分支指令。

6.7 函数调用

6.7.1 函数入口

遇到 FUNCTION 指令时生成函数标号。对于 main 函数，需要分配自己的栈帧（因为没有调用者）：

```

1. main:    addi $sp, $sp, -framesize

```

对于其他函数，栈帧由调用者在 CALL 指令处分配，函数入口不需要额外操

作。

6.7.2 参数传递与函数调用

函数调用分为几个步骤：

- 1 保存当前\$sp：用于从当前栈帧读取实参值
- 2 分配被调函数栈帧：addi \$sp, \$sp, -framesize
- 3 保存返回地址：sw \$ra, 0(\$sp)
- 4 传递参数：依次将实参从旧栈帧复制到新栈帧的形参位置
- 5 跳转调用：jal funcname
- 6 恢复返回地址：lw \$ra, 0(\$sp)
- 7 释放栈帧：addi \$sp, \$sp, framesize
- 8 保存返回值：将\$v0 中的返回值存入临时变量

6.7.3 函数返回

return 语句：将返回值加载到 v0 寄存器。对于 main 函数，执行系统调用 10 退出程序；对于其他函数，执行 jr \$ra 返回调用者。

6.7.4 内置函数 read 和 write

read 函数：输出提示信息（系统调用 4），然后读取整数（系统调用 5），返回值在\$v0 中。

write 函数：接收一个参数（在\$a0 中），输出整数（系统调用 1），然后输出换行（系统调用 4）。

这两个函数的实现附加在目标代码文件末尾。

6.8 目标代码示例

对于之前的示例程序，生成的 MIPS 汇编代码如下（部分）：

```
1. .data
2. _Prompt: .asciiz "Enter an integer: "
3. _ret: .asciiz "\n"
```

```

4. .text
5. .globl main
6.   j main
7.
8. main:
9.   addi $sp, $sp, -24
10.  li $t1, 5
11.  sw $t1, 12($sp)
12.  li $t1, 3
13.  sw $t1, 16($sp)
14.  li $t1, 2
15.  sw $t1, 24($sp)
16.  lw $t1, 16($sp)
17.  lw $t2, 24($sp)
18.  mul $t3, $t1, $t2
19.  sw $t3, 28($sp)
20.  lw $t1, 12($sp)
21.  lw $t2, 28($sp)
22.  add $t3, $t1, $t2
23.  sw $t3, 32($sp)
24.  lw $t1, 32($sp)
25.  sw $t1, 20($sp)
26.  lw $a0, 20($sp)
27.  addi $sp, $sp, -4
28.  sw $ra, 0($sp)
29.  jal write
30.  lw $ra, 0($sp)
31.  addi $sp, $sp, 4
32.  lw $v0, 36($sp)
33.  li $v0, 10
34.  syscall
35. exit_loop:
36.   j exit_loop
37.
38. write:
39.  li $v0, 1
40.  syscall
41.  li $v0, 4
42.  la $a0, _ret
43.  syscall
44.  move $v0, $0
45.  jr $ra

```

这段代码忠实地实现了原程序的语义：初始化变量、执行算术运算、调用输出函数、最后退出程序。

7 系统测试

7.1 概述

编译器的正确性验证需要经过系统性的测试，涵盖从词法分析到目标代码生成的各个阶段。本章详细介绍了 Mini-C 编译器的测试流程、测试方法和测试结果。

7.1.1 测试策略

编译器测试采用分阶段测试和端到端测试相结合的策略：

- 1 单元测试：独立测试各编译阶段的功能（词法、语法、语义、中间代码、目标代码）
- 2 功能测试：验证语言特性的完整实现（运算符、控制流、函数、数组等）
- 3 错误测试：验证编译器的错误检测和报告能力
- 4 综合测试：使用复杂程序测试各阶段的协同工作

7.1.2 测试环境

操作系统：Windows/Linux

编译工具：Flex 2.6.4, Bison 3.8.2, GCC/MinGW

目标模拟器：MARS

测试代码位置：testCodes/

7.1.3 编译流程

完整的命令如下，注意由于 Windows 的 Powershell 直接输出 utf-8 编码会出现中文乱码，因此加上了对应的参数以转换为 GBK 编码，可以在终端依次执行以下命令：

```
1. bison -d -o parser.tab.cpp parser.ypp
2. flex lex.l
3. g++ -std=c++11 -finput-charset=UTF-8 -fexec-charset=GBK -
   c parser.tab.cpp -o parser.tab.o
4. gcc -std=c11 -finput-charset=UTF-8 -fexec-charset=GBK -c lex.yy.c -
   o lex.yy.o
```

```
5. g++ -std=c++11 -finput-charset=UTF-8 -fexec-charset=GBK -c ast.cpp -  
   o ast.o  
6. g++ -std=c++11 -finput-charset=UTF-8 -fexec-charset=GBK -  
   c semantics.cpp -o semantics.o  
7. g++ -std=c++11 -finput-charset=UTF-8 -fexec-charset=GBK -c GenIR.cpp -  
   o GenIR.o  
8. g++ -std=c++11 -finput-charset=UTF-8 -fexec-charset=GBK -  
   c GenObject.cpp -o GenObject.o  
9. g++ -std=c++11 -o mini-  
   c.exe parser.tab.o lex.yy.o ast.o semantics.o GenIR.o GenObject.o
```

如果是在 Linux 上编译的就可以不需要中间额外的参数了，可以运行更简单的命令，我准备了一次性的命令脚本 `run.sh`，直接运行 `bash run.sh` 后即可得到编译程序，接下来运行命令 `./mini-c.exe + 待测代码路径` 即可运行程序。可以通过改变 `def.h` 文件中条件编译的内容来展示各种内容。

7.2 测试用例

本编译器使用三个测试文件来验证不同方面的功能。

7.2.1 test.c - 综合功能测试

文件路径：testCodes/test.c

测试目的：验证编译器的各项基本功能，包括变量、表达式、控制流、函数和数组。

覆盖的语言特性有

1. 全局变量声明
2. 函数定义与调用
3. 递归函数（斐波那契数列）
4. 条件语句（if-else）
5. while 循环
6. for 循环
7. 一维数组
8. 算术表达式与运算符优先级

9. 关系运算符与逻辑运算符

10. 内置函数 `write()`

7.2.2 error.c - 语义错误检测测试

文件路径: `testCodes/error.c`

测试目的: 验证编译器的语义错误检测能力。

包含的语义错误:

1. 第 5 行: 变量重复定义 (`int a;` 重复声明)
2. 第 7 行: 使用未定义变量 (`c` 未声明)
3. 第 8 行: 对非函数标识符使用函数调用 (`a(4)`, `a` 是变量不是函数)
4. 第 9 行: 对函数名使用非函数调用表达式 (`fibonacci=4`, 函数名不能赋值)
5. 第 10 行: 调用未定义函数 (`test` 未声明)
6. 第 11 行: 实参数量不匹配 (`fibonacci(3)` 传 1 个参数, 但声明需要 2 个)
7. 第 12 行: `break` 不在循环内
8. 第 13 行: `continue` 不在循环内
9. 第 30 行: 函数声明与定义参数数量不一致 (声明 2 个, 定义 1 个)
10. 第 23 行: 调用的函数 `jump` 只有声明没有定义

预期结果: 编译器应检测出所有 10 个语义错误, 并报告准确的行号和错误描述。

7.2.3 array.c - 多维数组测试

文件路径: `testCodes/array.c`

测试目的: 验证编译器对多维数组的支持, 包括数组声明、嵌套循环、数组元素访问和地址计算。

7.3 完整编译流程演示

本节以上面的测试用例展示从源代码到最终运行结果的完整编译过程, 包括词法分析、语法分析、语义分析、中间代码生成、目标代码生成五个阶段的输出。

7.3.1 词法分析与语法分析

test.c 的运行结果：

```
1.  外部变量定义：
2.      变量类型：int
3.
4.      变量列表：
5.          g_var
6.  外部变量定义：
7.      变量类型：float
8.
9.      变量列表：
10.         g_float
11. 函数定义：
12.      返回类型：int
13.
14.      函数名：fibo
15.      形参列表
16.          int
17.          n
18.      函数体：
19.          语句列表：
20.              if 语句：
21.                  条件：
22.                      ||
23.                      ==
24.                      n
25.
26.                      1
27.
28.
29.                      ==
30.                      n
31.
32.                      2
33.
34.
35.          满足条件执行：
36.              返回表达式：
37.                  1
38.
```

```

39.         返回表达式:
40.             +
41.         函数调用: 函数名: fibo
42.             1 个实参表达式:
43.
44.             -
45.             n
46.
47.             1
48.
49.
50.         函数调用: 函数名: fibo
51.             1 个实参表达式:
52.
53.             -
54.             n
55.
56.             2
57.
58.
59.
60.
61. 函数定义:
62.     返回类型: int
63.
64.     函数名: max
65.     形参列表
66.         int
67.         a
68.         int
69.         b
70.     函数体:
71.         语句列表:
72.             if-else 语句:
73.                 条件:
74.                     >
75.                     a
76.
77.                     b
78.
79.
80.         满足条件执行:
81.             返回表达式:
82.                 a
83.

```

```

84.         否则执行：
85.         返回表达式：
86.         b
87.
88.
89. 函数定义：
90.     返回类型：int
91.
92.     函数名：main
93.     形参列表（无参数）
94.     函数体：
95.         局部变量定义：
96.         变量类型：int
97.
98.         变量声明：
99.         a
100.        b
101.        c
102.        i
103.        result
104.
105.        变量类型：int
106.
107.        变量声明：
108.        nums[10]
109.
110.        语句列表：
111.        表达式语句：
112.            赋值表达式，运算符：=
113.            左值表达式：
114.            a
115.            右值表达式：
116.            5
117.
118.        表达式语句：
119.            赋值表达式，运算符：=
120.            左值表达式：
121.            b
122.            右值表达式：
123.            3
124.
125.        表达式语句：
126.            赋值表达式，运算符：=
127.            左值表达式：
128.            c

```

129.	右值表达式:
130.	+
131.	a
132.	
133.	*
134.	b
135.	
136.	2
137.	
138.	
139.	
140.	函数调用: 函数名: write
141.	1 个实参表达式:
142.	
143.	c
144.	表达式语句:
145.	赋值表达式, 运算符: =
146.	左值表达式:
147.	a
148.	右值表达式:
149.	10
150.	
151.	表达式语句:
152.	赋值表达式, 运算符: =
153.	左值表达式:
154.	b
155.	右值表达式:
156.	20
157.	
158.	表达式语句:
159.	赋值表达式, 运算符: =
160.	左值表达式:
161.	c
162.	右值表达式:
163.	函数调用: 函数名: max
164.	2 个实参表达式:
165.	
166.	a
167.	b
168.	
169.	函数调用: 函数名: write
170.	1 个实参表达式:
171.	
172.	c
173.	表达式语句:

```

174.          赋值表达式, 运算符: =
175.          左值表达式:
176.                  i
177.          右值表达式:
178.                  1
179.
180.          表达式语句:
181.          赋值表达式, 运算符: =
182.          左值表达式:
183.                  result
184.          右值表达式:
185.                  0
186.
187.          while 语句:
188.          循环条件:
189.                  <=
190.                  i
191.
192.                  5
193.
194.          循环体:
195.          复合语句:
196.          语句列表:
197.          表达式语句:
198.          赋值表达式, 运算符: =
199.          左值表达式:
200.                  result
201.          右值表达式:
202.                  +
203.                  result
204.
205.                  i
206.
207.
208.          表达式语句:
209.          赋值表达式, 运算符: =
210.          左值表达式:
211.                  i
212.          右值表达式:
213.                  +
214.                  i
215.
216.                  1
217.
218.

```

219.	函数调用：函数名：write
220.	1 个实参表达式：
221.	
222.	result
223.	表达式语句：
224.	赋值表达式，运算符：=
225.	左值表达式：
226.	result
227.	右值表达式：
228.	0
229.	
230.	for 语句：
231.	初始化表达式：
232.	赋值表达式，运算符：=
233.	左值表达式：
234.	i
235.	右值表达式：
236.	0
237.	循环条件：
238.	<
239.	i
240.	
241.	10
242.	
243.	迭代表达式：
244.	赋值表达式，运算符：=
245.	左值表达式：
246.	i
247.	右值表达式：
248.	+
249.	i
250.	
251.	1
252.	
253.	循环体：
254.	复合语句：
255.	语句列表：
256.	表达式语句：
257.	赋值表达式，运算符：=
258.	左值表达式：
259.	nums
260.	1 个下标表达式：
261.	
262.	i

263.	右值表达式:
264.	i
265.	
266.	表达式语句:
267.	赋值表达式, 运算符: =
268.	左值表达式:
269.	result
270.	右值表达式:
271.	+
272.	result
273.	
274.	nums
275.	1 个下标表
	达式:
276.	
277.	i
278.	
279.	
280.	函数调用: 函数名: write
281.	1 个实参表达式:
282.	
283.	result
284.	表达式语句:
285.	赋值表达式, 运算符: =
286.	左值表达式:
287.	result
288.	右值表达式:
289.	函数调用: 函数名: fibo
290.	1 个实参表达式:
291.	
292.	6
293.	
294.	函数调用: 函数名: write
295.	1 个实参表达式:
296.	
297.	result
298.	表达式语句:
299.	赋值表达式, 运算符: =
300.	左值表达式:
301.	a
302.	右值表达式:
303.	10
304.	
305.	表达式语句:
306.	赋值表达式, 运算符: =

307.	左值表达式:
308.	b
309.	右值表达式:
310.	5
311.	
312.	表达式语句:
313.	赋值表达式, 运算符: =
314.	左值表达式:
315.	c
316.	右值表达式:
317.	-
318.	*
319.	+
320.	a
321.	
322.	b
323.	
324.	
325.	2
326.	
327.	
328.	/
329.	a
330.	
331.	b
332.	
333.	
334.	
335.	函数调用: 函数名: write
336.	1 个实参表达式:
337.	
338.	c
339.	返回表达式:
340.	0

array.c 的运行结果:

1.	函数定义:
2.	返回类型: int
3.	
4.	函数名: main
5.	形参列表 (无参数)
6.	函数体:
7.	局部变量定义:
8.	变量类型: int

```

9.
10.      变量声明:
11.          arr[3][4]
12.
13.      变量类型: int
14.
15.      变量声明:
16.          i
17.          j
18.          sum
19.
20.      语句列表:
21.      for 语句:
22.          初始化表达式:
23.              赋值表达式, 运算符: =
24.              左值表达式:
25.                  i
26.              右值表达式:
27.                  0
28.          循环条件:
29.              <
30.              i
31.
32.              3
33.
34.          迭代表达式:
35.              赋值表达式, 运算符: =
36.              左值表达式:
37.                  i
38.              右值表达式:
39.                  +
40.                  i
41.
42.                  1
43.
44.      循环体:
45.          复合语句:
46.          语句列表:
47.          for 语句:
48.              初始化表达式:
49.                  赋值表达式, 运算符: =
50.                  左值表达式:
51.                      j
52.                  右值表达式:
53.                      0

```

54.	循环条件:
55.	<
56.	j
57.	
58.	4
59.	
60.	迭代表达式:
61.	赋值表达式, 运算符: =
62.	左值表达式:
63.	j
64.	右值表达式:
65.	+
66.	j
67.	
68.	1
69.	
70.	循环体:
71.	复合语句:
72.	语句列表:
73.	表达式语句:
74.	赋值表达式, 运算符: =
75.	左值表达式:
76.	arr
77.	2 个下
	标表达式:
78.	
79.	i
80.	j
81.	右值表达式:
82.	+
83.	*
84.	i
85.	
86.	4
87.	
88.	
89.	j
90.	
91.	
92.	表达式语句:
93.	赋值表达式, 运算符: =
94.	左值表达式:
95.	sum

96.	右值表达式:
97.	0
98.	
99.	for 语句:
100.	初始化表达式:
101.	赋值表达式, 运算符: =
102.	左值表达式:
103.	i
104.	右值表达式:
105.	0
106.	循环条件:
107.	<
108.	i
109.	
110.	3
111.	
112.	迭代表达式:
113.	赋值表达式, 运算符: =
114.	左值表达式:
115.	i
116.	右值表达式:
117.	+
118.	i
119.	
120.	1
121.	
122.	循环体:
123.	复合语句:
124.	语句列表:
125.	for 语句:
126.	初始化表达式:
127.	赋值表达式, 运算符: =
128.	左值表达式:
129.	j
130.	右值表达式:
131.	0
132.	循环条件:
133.	<
134.	j
135.	
136.	4
137.	
138.	迭代表达式:
139.	赋值表达式, 运算符: =
140.	左值表达式:

```

141.                                     j
142.                                右值表达式:
143.                                     +
144.                                     j
145.
146.                                     1
147.
148.                                循环体:
149.                                复合语句:
150.                                语句列表:
151.                                表达式语句:
152.                                赋值表达式, 运算符: =
153.                                左值表达式:
154.                                     sum
155.                                右值表达式:
156.                                     +
157.                                     sum
158.
159.                                     arr
160.
    2 个下标表达式:
161.
162.                                     i
163.                                     j
164.
165.
166.                                函数调用: 函数名: write
167.                                1 个实参表达式:
168.
169.                                     sum
170.                                返回表达式:
171.                                     0

```

可以由结果判断程序能正确识别关键字（int, if, return, for 等）、正确识别标识符和常量、支持多字符运算符正确匹配（==, ||, <= 等）并能准确跟踪行号和列号。

7.3.2 语义分析

test.c:

```

1. ===== 开始语义分析 =====
2.
3. [符号表] 创建全局符号表 (层次 0)

```

```

4. [符号表] 添加预定义函数: int read()
5. [符号表] 添加预定义函数: int write(int x)
6.
7. 初始符号表 (包含预定义函数):
8.
9. =====
10.                                符号表内容
11. =====
12. -----
13. 层次: 0 (全局符号表)
14. 符 号 名          别名      类型      种 类      其他信息
15. -----
16. read                                int      函数      参数个数:0 AR 大小:12 [已定
    义]
17. write                                int      函数      参数个数:1 AR 大小:4 [已定
    义]
18. x                x          int      参数      偏移量:4 全局:0
19. -----
20. =====
21.
22.
23. [语义分析] 开始处理外部定义...
24. [语义分析] 处理外部变量定义
25. [符号表] 添加全局变量: int g_var (别名:V_1, 偏移:0)
26. [语义分析] 处理外部变量定义
27. [符号表] 添加全局变量: float g_float (别名:V_2, 偏移:0)
28.
29. [符号表] 定义函数: int fibo() - 参数个数:1
30. [符号表] 进入函数 fibo 的作用域 (层次 1)
31. [符号表] 添加函数参数: int n (别名:V_3, 偏移:12)
32.
33. >>> 当前复合语句符号表状态 <<<
34.
35. =====
36.                                符号表内容
37. =====
38. -----
39. 层次: 0 (全局符号表)
40. 符 号 名          别名      类型      种 类      其他信息
41. -----
42. read                                int      函数      参数个数:0 AR 大小:12 [已定
    义]
43. write                                int      函数      参数个数:1 AR 大小:4 [已定
    义]
44. x                x          int      参数      偏移量:4 全局:0

```

```

45. g_var          V_1      int      变量      偏移量:0 全局:1
46. g_float        V_2      float     变量      偏移量:0 全局:1
47. fibo           int      函数      参数个数:1 AR 大小:0 [已定
    义]
48. -----
49. -----
50. 层次: 1 (局部符号表)
51. 符号名          别名      类型      种 类      其他信息
52. -----
53. n                V_3      int      参数      偏移量:12 全局:0
54. -----
55. =====
56.
57. [符号表] 离开复合语句作用域 (层次 1)
58.
59. [符号表] 函数 fibo AR 大小: 16
60.
61. [符号表] 定义函数: int max() - 参数个数:2
62. [符号表] 进入函数 max 的作用域 (层次 1)
63. [符号表] 添加函数参数: int a (别名:V_4, 偏移:12)
64. [符号表] 添加函数参数: int b (别名:V_5, 偏移:16)
65.
66. >>> 当前复合语句符号表状态 <<<
67.
68. =====
69.                                符号表内容
70. =====
71. -----
72. 层次: 0 (全局符号表)
73. 符号名          别名      类型      种 类      其他信息
74. -----
75. read            int      函数      参数个数:0 AR 大小:12 [已定
    义]
76. write           int      函数      参数个数:1 AR 大小:4 [已定
    义]
77. x                x        int      参数      偏移量:4 全局:0
78. g_var          V_1      int      变量      偏移量:0 全局:1
79. g_float        V_2      float     变量      偏移量:0 全局:1
80. fibo           int      函数      参数个数:1 AR 大小:16 [已定
    义]
81. max            int      函数      参数个数:2 AR 大小:0 [已定
    义]
82. -----
83. -----
84. 层次: 1 (局部符号表)

```

```

85. 符号名          别名      类型      种 类      其他信息
86. -----
87. a                V_4      int       参数       偏移量:12 全局:0
88. b                V_5      int       参数       偏移量:16 全局:0
89. -----
90. =====
91.
92. [符号表] 离开复合语句作用域 (层次 1)
93.
94. [符号表] 函数 max AR 大小: 20
95.
96. [符号表] 定义函数: int main() - 参数个数:0
97. [符号表] 进入函数 main 的作用域 (层次 1)
98. [符号表] 添加局部变量: int a (别名:V_6, 偏移:12)
99. [符号表] 添加局部变量: int b (别名:V_7, 偏移:16)
100. [符号表] 添加局部变量: int c (别名:V_8, 偏移:20)
101. [符号表] 添加局部变量: int i (别名:V_9, 偏移:24)
102. [符号表] 添加局部变量: int result (别名:V_10, 偏移:28)
103. [符号表] 添加局部数组: int nums[10] (别名:V_11, 偏移:32)
104. [符号表] 进入新的复合语句作用域 (层次 2)
105.
106. >>> 当前复合语句符号表状态 <<<
107.
108. =====
109.                                     符号表内容
110. =====
111. -----
112. 层次: 0 (全局符号表)
113. 符号名          别名      类型      种 类      其他信息
114. -----
115.  read                int       函数       参数个数:0 AR 大小:12 [已
    定义]
116.  write                int       函数       参数个数:1 AR 大小:4 [已
    定义]
117.  x                    x         int       参数       偏移量:4 全局:0
118.  g_var                V_1      int       变量       偏移量:0 全局:1
119.  g_float              V_2      float     变量       偏移量:0 全局:1
120.  fibo                  int       函数       参数个数:1 AR 大小:16 [已
    定义]
121.  max                  int       函数       参数个数:2 AR 大小:20 [已
    定义]

```

```

122.  main                int      函数      参数个数:0 AR 大小:0 [已
      定义]
123.  -----
      --
124.  -----
      --
125.  层次: 1 (局部符号表)
126.  符 号 名            别名      类型      种 类      其他信息
127.  -----
      --
128.  a                  V_6       int      变量      偏移量:12 全局:0
129.  b                  V_7       int      变量      偏移量:16 全局:0
130.  c                  V_8       int      变量      偏移量:20 全局:0
131.  i                  V_9       int      变量      偏移量:24 全局:0
132.  result            V_10      int      变量      偏移量:28 全局:0
133.  nums              V_11      int      数组      偏移量:32 全局:0 1 维数组
      [10]
134.  -----
      --
135.  -----
      --
136.  层次: 2 (局部符号表)
137.  符 号 名            别名      类型      种 类      其他信息
138.  -----
      --
139.  空 表
140.  -----
      --
141.  =====
      ==
142.
143.  [符号表] 离开复合语句作用域 (层次 2)
144.
145.  [符号表] 进入新的复合语句作用域 (层次 2)
146.
147.  >>> 当前复合语句符号表状态 <<<
148.
149.  =====
      ==
150.                                符号表内容
151.  =====
      ==
152.  -----
      --
153.  层次: 0 (全局符号表)

```

154.	符 号 名	别 名	类 型	种 类	其他信息
155.	-----				
	--				
156.	read		int	函数	参数个数:0 AR 大小:12 [已定义]
157.	write		int	函数	参数个数:1 AR 大小:4 [已定义]
158.	x	x	int	参数	偏移量:4 全局:0
159.	g_var	V_1	int	变量	偏移量:0 全局:1
160.	g_float	V_2	float	变量	偏移量:0 全局:1
161.	fibo		int	函数	参数个数:1 AR 大小:16 [已定义]
162.	max		int	函数	参数个数:2 AR 大小:20 [已定义]
163.	main		int	函数	参数个数:0 AR 大小:0 [已定义]
164.	-----				
	--				
165.	-----				
	--				
166.	层次: 1 (局部符号表)				
167.	符 号 名	别 名	类 型	种 类	其他信息
168.	-----				
	--				
169.	a	V_6	int	变量	偏移量:12 全局:0
170.	b	V_7	int	变量	偏移量:16 全局:0
171.	c	V_8	int	变量	偏移量:20 全局:0
172.	i	V_9	int	变量	偏移量:24 全局:0
173.	result	V_10	int	变量	偏移量:28 全局:0
174.	nums	V_11	int	数组	偏移量:32 全局:0 1 维数组 [10]
175.	-----				
	--				
176.	-----				
	--				
177.	层次: 2 (局部符号表)				
178.	符 号 名	别 名	类 型	种 类	其他信息
179.	-----				
	--				
180.	空 表				
181.	-----				
	--				
182.	=====				
	==				
183.					

```

184.  [符号表] 离开复合语句作用域（层次 2）
185.
186.
187.  >>> 当前复合语句符号表状态 <<<
188.
189.  =====
    ==
190.                      符号表内容
191.  =====
    ==
192.  -----
    --
193.  层次: 0 (全局符号表)
194.  符 号 名          别名      类型      种 类      其他信息
195.  -----
    --
196.  read                      int      函数      参数个数:0 AR 大小:12 [已
    定义]
197.  write                      int      函数      参数个数:1 AR 大小:4 [已
    定义]
198.  x                        x      int      参数      偏移量:4 全局:0
199.  g_var                     V_1    int      变量      偏移量:0 全局:1
200.  g_float                   V_2    float    变量      偏移量:0 全局:1
201.  fibo                      int      函数      参数个数:1 AR 大小:16 [已
    定义]
202.  max                      int      函数      参数个数:2 AR 大小:20 [已
    定义]
203.  main                      int      函数      参数个数:0 AR 大小:0 [已
    定义]
204.  -----
    --
205.  -----
    --
206.  层次: 1 (局部符号表)
207.  符 号 名          别名      类型      种 类      其他信息
208.  -----
    --
209.  a                        V_6    int      变量      偏移量:12 全局:0
210.  b                        V_7    int      变量      偏移量:16 全局:0
211.  c                        V_8    int      变量      偏移量:20 全局:0
212.  i                        V_9    int      变量      偏移量:24 全局:0
213.  result                   V_10   int      变量      偏移量:28 全局:0
214.  nums                     V_11   int      数组      偏移量:32 全局:0 1 维数组
    [10]

```

```

215. -----
    --
216. =====
    ==
217.
218. [符号表] 离开复合语句作用域（层次 1）
219.
220. [符号表] 函数 main AR 大小: 72
221.
222. 最终全局符号表:
223.
224. =====
    ==
225.                                符号表内容
226. =====
    ==
227. -----
    --
228. 层次: 0 (全局符号表)
229. 符 号 名          别名      类型      种 类      其他信息
230. -----
    --
231. read                                int      函数      参数个数:0 AR 大小:12 [已
    定义]
232. write                                int      函数      参数个数:1 AR 大小:4 [已
    定义]
233. x                x                int      参数      偏移量:4 全局:0
234. g_var            V_1              int      变量      偏移量:0 全局:1
235. g_float          V_2              float     变量      偏移量:0 全局:1
236. fibo                                int      函数      参数个数:1 AR 大小:16 [已
    定义]
237. max                                int      函数      参数个数:2 AR 大小:20 [已
    定义]
238. main                                int      函数      参数个数:0 AR 大小:72 [已
    定义]
239. -----
    --
240. =====
    ==
241.
242.
243. ===== 语义分析完成 =====
244.
245. [成功] 未发现语义错误, 程序语义正确!

```

error.c:

```
1. ===== 开始语义分析 =====
2.
3. [符号表] 创建全局符号表 (层次 0)
4. [符号表] 添加预定义函数: int read()
5. [符号表] 添加预定义函数: int write(int x)
6.
7. 初始符号表 (包含预定义函数):
8.
9. =====
10.                                符号表内容
11. =====
12. -----
13. 层次: 0 (全局符号表)
14. 符 号 名          别名      类型      种 类      其他信息
15. -----
16. read                                int      函数      参数个数:0 AR 大小:12 [已定
   义]
17. write                                int      函数      参数个数:1 AR 大小:4 [已定
   义]
18. x                                x      int      参数      偏移量:4 全局:0
19. -----
20. =====
21.
22.
23. [语义分析] 开始处理外部定义...
24. [符号表] 声明函数: int fibo() - 参数个数:2
25. [符号表] 声明函数: int jump() - 参数个数:1
26.
27. [符号表] 定义函数: int main() - 参数个数:0
28. [符号表] 进入函数 main 的作用域 (层次 1)
29. [符号表] 添加局部变量: int a (别名:V_1, 偏移:12)
30. [符号表] 添加局部变量: int b (别名:V_2, 偏移:16)
31. [符号表] 进入新的复合语句作用域 (层次 2)
32.
33. >>> 当前复合语句符号表状态 <<<
34.
35. =====
36.                                符号表内容
37. =====
38. -----
39. 层次: 0 (全局符号表)
40. 符 号 名          别名      类型      种 类      其他信息
41. -----
```

42.	read		int	函数	参数个数:0 AR 大小:12 [已定义]
43.	write		int	函数	参数个数:1 AR 大小:4 [已定义]
44.	x	x	int	参数	偏移量:4 全局:0
45.	fibo		int	函数	参数个数:2 AR 大小:12 [仅声明]
46.	jump		int	函数	参数个数:1 AR 大小:12 [仅声明]
47.	main		int	函数	参数个数:0 AR 大小:0 [已定义]
48.	-----				
49.	-----				
50.	层次: 1 (局部符号表)				
51.	符 号 名	别名	类型	种 类	其他信息
52.	-----				
53.	a	V_1	int	变量	偏移量:12 全局:0
54.	b	V_2	int	变量	偏移量:16 全局:0
55.	-----				
56.	-----				
57.	层次: 2 (局部符号表)				
58.	符 号 名	别名	类型	种 类	其他信息
59.	-----				
60.	空 表				
61.	-----				
62.	=====				
63.					
64.	[符号表] 离开复合语句作用域 (层次 2)				
65.					
66.	[符号表] 进入新的复合语句作用域 (层次 2)				
67.					
68.	>>> 当前复合语句符号表状态 <<<				
69.					
70.	=====				
71.	符号表内容				
72.	=====				
73.	-----				
74.	层次: 0 (全局符号表)				
75.	符 号 名	别名	类型	种 类	其他信息
76.	-----				
77.	read		int	函数	参数个数:0 AR 大小:12 [已定义]
78.	write		int	函数	参数个数:1 AR 大小:4 [已定义]
79.	x	x	int	参数	偏移量:4 全局:0

80.	fibo		int	函数	参数个数:2 AR 大小:12 [仅声明]
81.	jump		int	函数	参数个数:1 AR 大小:12 [仅声明]
82.	main		int	函数	参数个数:0 AR 大小:0 [已定义]
83.	-----				
84.	-----				
85.	层次: 1 (局部符号表)				
86.	符 号 名	别名	类型	种 类	其他信息
87.	-----				
88.	a	V_1	int	变量	偏移量:12 全局:0
89.	b	V_2	int	变量	偏移量:16 全局:0
90.	-----				
91.	-----				
92.	层次: 2 (局部符号表)				
93.	符 号 名	别名	类型	种 类	其他信息
94.	-----				
95.	空 表				
96.	-----				
97.	=====				
98.					
99.	[符号表] 离开复合语句作用域 (层次 2)				
100.					
101.	[符号表] 进入新的复合语句作用域 (层次 2)				
102.					
103.	>>> 当前复合语句符号表状态 <<<				
104.					
105.	=====				
	==				
106.	符号表内容				
107.	=====				
	==				
108.	-----				
	--				
109.	层次: 0 (全局符号表)				
110.	符 号 名	别名	类型	种 类	其他信息
111.	-----				
	--				
112.	read		int	函数	参数个数:0 AR 大小:12 [已定义]
113.	write		int	函数	参数个数:1 AR 大小:4 [已定义]
114.	x	x	int	参数	偏移量:4 全局:0

```

115.  fibo                                int      函数      参数个数:2 AR 大小:12 [仅
      声明]
116.  jump                                int      函数      参数个数:1 AR 大小:12 [仅
      声明]
117.  main                                int      函数      参数个数:0 AR 大小:0 [已
      定义]
118.  -----
      --
119.  -----
      --
120.  层次: 1 (局部符号表)
121.  符 号 名          别名      类型      种 类      其他信息
122.  -----
      --
123.  a                  V_1      int      变量      偏移量:12 全局:0
124.  b                  V_2      int      变量      偏移量:16 全局:0
125.  -----
      --
126.  -----
      --
127.  层次: 2 (局部符号表)
128.  符 号 名          别名      类型      种 类      其他信息
129.  -----
      --
130.  空 表
131.  -----
      --
132.  =====
      ==
133.
134.  [符号表] 离开复合语句作用域 (层次 2)
135.
136.
137.  >>> 当前复合语句符号表状态 <<<
138.
139.  =====
      ==
140.                                符号表内容
141.  =====
      ==
142.  -----
      --
143.  层次: 0 (全局符号表)
144.  符 号 名          别名      类型      种 类      其他信息

```

```

145. -----
    --
146.  read                      int      函数      参数个数:0 AR 大小:12 [已
      定义]
147.  write                    int      函数      参数个数:1 AR 大小:4 [已
      定义]
148.  x                        x        int      参数      偏移量:4 全局:0
149.  fibo                     int      函数      参数个数:2 AR 大小:12 [仅
      声明]
150.  jump                    int      函数      参数个数:1 AR 大小:12 [仅
      声明]
151.  main                    int      函数      参数个数:0 AR 大小:0 [已
      定义]
152. -----
    --
153. -----
    --
154.  层次: 1 (局部符号表)
155.  符 号 名                别名      类型      种 类      其他信息
156.  -----
    --
157.  a                      V_1      int      变量      偏移量:12 全局:0
158.  b                      V_2      int      变量      偏移量:16 全局:0
159.  -----
    --
160.  =====
    ==
161.
162.  [符号表] 离开复合语句作用域 (层次 1)
163.
164.  [符号表] 函数 main AR 大小: 20
165.
166.  [符号表] 为先前声明的函数 fibo 提供定义
167.  [符号表] 进入函数 fibo 的作用域 (层次 1)
168.  [符号表] 添加函数参数: int a (别名:V_3, 偏移:12)
169.
170.  >>> 当前复合语句符号表状态 <<<
171.
172.  =====
    ==
173.                                符号表内容
174.  =====
    ==
175. -----
    --

```

176.	层次：0（全局符号表）				
177.	符 号 名	别名	类型	种 类	其他信息
178.	-----				
	--				
179.	read		int	函数	参数个数:0 AR 大小:12 [已定义]
180.	write		int	函数	参数个数:1 AR 大小:4 [已定义]
181.	x	x	int	参数	偏移量:4 全局:0
182.	fibo		int	函数	参数个数:1 AR 大小:12 [仅声明]
183.	jump		int	函数	参数个数:1 AR 大小:12 [仅声明]
184.	main		int	函数	参数个数:0 AR 大小:20 [已定义]
185.	-----				
	--				
186.	-----				
	--				
187.	层次：1（局部符号表）				
188.	符 号 名	别名	类型	种 类	其他信息
189.	-----				
	--				
190.	a	V_3	int	参数	偏移量:12 全局:0
191.	-----				
	--				
192.	=====				
	==				
193.					
194.	[符号表] 离开复合语句作用域（层次1）				
195.					
196.	[符号表] 函数 fibo AR 大小: 16				
197.					
198.	最终全局符号表:				
199.					
200.	=====				
	==				
201.	符号表内容				
202.	=====				
	==				
203.	-----				
	--				
204.	层次：0（全局符号表）				
205.	符 号 名	别名	类型	种 类	其他信息

```

206.  -----
    --
207.  read                                int      函数      参数个数:0 AR 大小:12 [已
      定义]
208.  write                              int      函数      参数个数:1 AR 大小:4 [已
      定义]
209.  x                                x          int      参数      偏移量:4 全局:0
210.  fibo                              int      函数      参数个数:1 AR 大小:16 [已
      定义]
211.  jump                              int      函数      参数个数:1 AR 大小:12 [仅
      声明]
212.  main                              int      函数      参数个数:0 AR 大小:20 [已
      定义]
213.  -----
    --
214.  =====
    ==
215.
216.
217.  ===== 语义分析完成 =====
218.
219.
220.  发现以下语义错误:
221.
222.  第 5 行, 第 9 列发生错误: 变量 a 重复定义
223.  第 7 行, 第 7 列发生错误: 使用未定义的符号 c
224.  第 8 行, 第 8 列发生错误: 对非函数标识符使用函数调用表达式
225.  第 9 行, 第 9 列发生错误: 对函数名使用非函数调用表达式形式
226.  第 10 行, 第 11 列发生错误: 调用未定义的函数 test
227.  第 11 行, 第 11 列发生错误: 实参表达式数量与形参不一致
228.  第 12 行, 第 10 列发生错误: break 语句不在循环语句内
229.  第 13 行, 第 13 列发生错误: continue 语句不在循环语句内
230.  第 30 行, 第 1 列发生错误: 函数 fibo 声明与定义的参数数量不同
231.  第 23 行, 第 11 列发生错误: 调用的函数 jump 未定义

```

可见都能生成相应的符号表，而且对于正确的 test.c 代码语义分析会通过，对于有错的 error.c 则会正确检测出 10 个语义错误并报告详细的错误信息。

7.3.3 中间代码生成

test.c:

```

1. FUNCTION fibo:
2.   PARAM n

```

```

3.   Temp_1 := #1
4.   IF V_3==Temp_1 GOTO Label_1
5.   GOTO Label_3
6. LABEL Label_3:
7.   Temp_2 := #2
8.   IF V_3==Temp_2 GOTO Label_1
9.   GOTO Label_2
10. LABEL Label_1:
11.  Temp_3 := #1
12.  RETURN Temp_3
13. LABEL Label_2:
14.  Temp_4 := #1
15.  Temp_5:= V_3-Temp_4
16.  ARG Temp_5
17.  Temp_6 := CALL fibo
18.  Temp_7 := #2
19.  Temp_8:= V_3-Temp_7
20.  ARG Temp_8
21.  Temp_9 := CALL fibo
22.  Temp_10:= Temp_6+Temp_9
23.  RETURN Temp_10
24. FUNCTION max:
25.  PARAM a
26.  PARAM b
27.  IF V_4>V_5 GOTO Label_4
28.  GOTO Label_5
29. LABEL Label_4:
30.  RETURN V_4
31.  GOTO Label_6
32. LABEL Label_5:
33.  RETURN V_5
34. LABEL Label_6:
35. FUNCTION main:
36.  Temp_11 := #5
37.  V_6 := Temp_11
38.  Temp_12 := #3
39.  V_7 := Temp_12
40.  Temp_13 := #2
41.  Temp_14:= V_7*Temp_13
42.  Temp_15:= V_6+Temp_14
43.  V_8 := Temp_15
44.  ARG V_8
45.  Temp_16 := CALL write
46.  Temp_17 := #10
47.  V_6 := Temp_17

```

```

48. Temp_18 := #20
49. V_7 := Temp_18
50. ARG V_6
51. ARG V_7
52. Temp_19 := CALL max
53. V_8 := Temp_19
54. ARG V_8
55. Temp_20 := CALL write
56. Temp_21 := #1
57. V_9 := Temp_21
58. Temp_22 := #0
59. V_10 := Temp_22
60. LABEL Label_7:
61. Temp_23 := #5
62. IF V_9<=Temp_23 GOTO Label_8
63. GOTO Label_9
64. LABEL Label_8:
65. Temp_24:= V_10+V_9
66. V_10 := Temp_24
67. Temp_25 := #1
68. Temp_26:= V_9+Temp_25
69. V_9 := Temp_26
70. GOTO Label_7
71. LABEL Label_9:
72. ARG V_10
73. Temp_27 := CALL write
74. Temp_28 := #0
75. V_10 := Temp_28
76. Temp_29 := #0
77. V_9 := Temp_29
78. LABEL Label_10:
79. Temp_30 := #10
80. IF V_9<Temp_30 GOTO Label_11
81. GOTO Label_12
82. LABEL Label_11:
83. Temp_33 := #4
84. Temp_34:= V_9*Temp_33
85. V_11[Temp_34]:=V_9
86. Temp_35 := #4
87. Temp_36:= V_9*Temp_35
88. Temp_37:=V_11[Temp_36]
89. Temp_38:= V_10+Temp_37
90. V_10 := Temp_38
91. LABEL Label_13:
92. Temp_31 := #1

```

```

93.  Temp_32:= V_9+Temp_31
94.  V_9 := Temp_32
95.  GOTO Label_10
96. LABEL Label_12:
97.  ARG V_10
98.  Temp_39 := CALL write
99.  Temp_40 := #6
100.  ARG Temp_40
101.  Temp_41 := CALL fibo
102.  V_10 := Temp_41
103.  ARG V_10
104.  Temp_42 := CALL write
105.  Temp_43 := #10
106.  V_6 := Temp_43
107.  Temp_44 := #5
108.  V_7 := Temp_44
109.  Temp_45:= V_6+V_7
110.  Temp_46 := #2
111.  Temp_47:= Temp_45*Temp_46
112.  Temp_48:= V_6/V_7
113.  Temp_49:= Temp_47-Temp_48
114.  V_8 := Temp_49
115.  ARG V_8
116.  Temp_50 := CALL write
117.  Temp_51 := #0
118.  RETURN Temp_51
119.  End Of Program

```

array.c:

```

1. FUNCTION main:
2.  Temp_1 := #0
3.  V_2 := Temp_1
4. LABEL Label_1:
5.  Temp_2 := #3
6.  IF V_2<Temp_2 GOTO Label_2
7.  GOTO Label_3
8. LABEL Label_2:
9.  Temp_5 := #0
10. V_3 := Temp_5
11. LABEL Label_5:
12.  Temp_6 := #4
13.  IF V_3<Temp_6 GOTO Label_6
14.  GOTO Label_7
15. LABEL Label_6:

```

```

16. Temp_9 := #4
17. Temp_10:= V_2*Temp_9
18. Temp_11:= Temp_10+V_3
19. Temp_12 := #4
20. Temp_13:= Temp_11*Temp_12
21. Temp_14 := #4
22. Temp_15:= V_2*Temp_14
23. Temp_16:= Temp_15+V_3
24. V_1[Temp_13]:=Temp_16
25. LABEL Label_8:
26. Temp_7 := #1
27. Temp_8:= V_3+Temp_7
28. V_3 := Temp_8
29. GOTO Label_5
30. LABEL Label_7:
31. LABEL Label_4:
32. Temp_3 := #1
33. Temp_4:= V_2+Temp_3
34. V_2 := Temp_4
35. GOTO Label_1
36. LABEL Label_3:
37. Temp_17 := #0
38. V_4 := Temp_17
39. Temp_18 := #0
40. V_2 := Temp_18
41. LABEL Label_9:
42. Temp_19 := #3
43. IF V_2<Temp_19 GOTO Label_10
44. GOTO Label_11
45. LABEL Label_10:
46. Temp_22 := #0
47. V_3 := Temp_22
48. LABEL Label_13:
49. Temp_23 := #4
50. IF V_3<Temp_23 GOTO Label_14
51. GOTO Label_15
52. LABEL Label_14:
53. Temp_26 := #4
54. Temp_27:= V_2*Temp_26
55. Temp_28:= Temp_27+V_3
56. Temp_29 := #4
57. Temp_30:= Temp_28*Temp_29
58. Temp_31:=V_1[Temp_30]
59. Temp_32:= V_4+Temp_31
60. V_4 := Temp_32

```

```

61. LABEL Label_16:
62.   Temp_24 := #1
63.   Temp_25:= V_3+Temp_24
64.   V_3 := Temp_25
65.   GOTO   Label_13
66. LABEL Label_15:
67. LABEL Label_12:
68.   Temp_20 := #1
69.   Temp_21:= V_2+Temp_20
70.   V_2 := Temp_21
71.   GOTO   Label_9
72. LABEL Label_11:
73.   ARG   V_4
74.   Temp_33 := CALL write
75.   Temp_34 := #0
76.   RETURN Temp_34
77. End Of Program

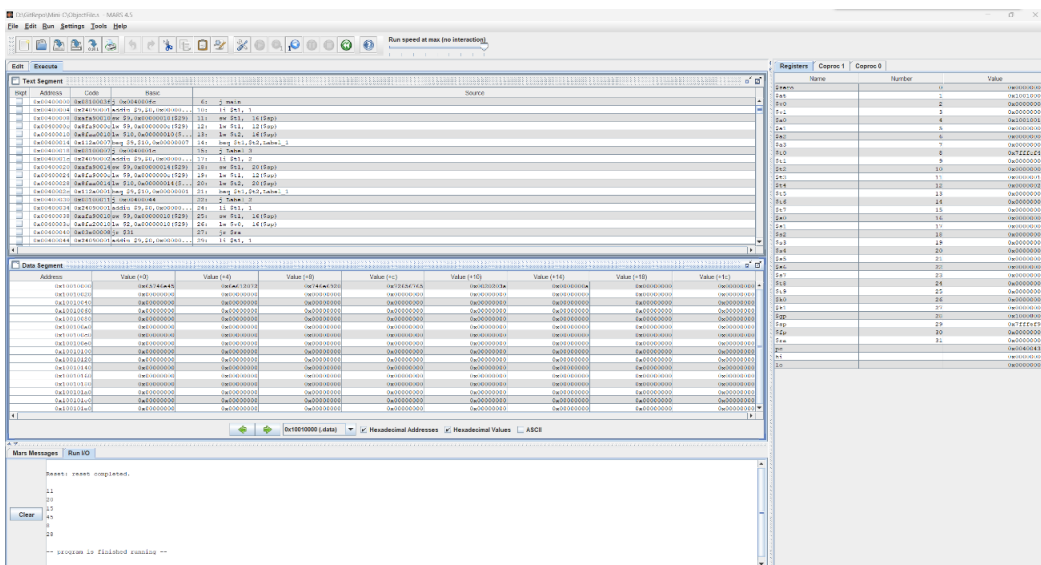
```

可见都能生成正确的中间代码，运算符优先级正确体现，控制流正确，函数调用机制正确（ARG、CALL、RETURN），数组访问偏移量计算正确，短路求值实现，符合我们的预期

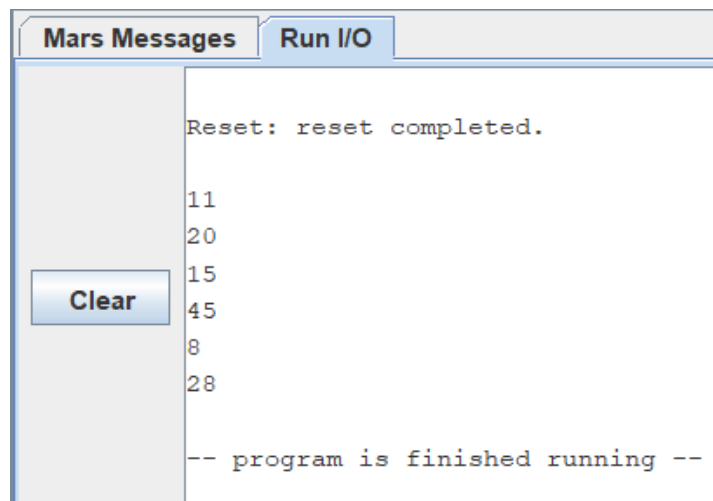
7.3.4 目标代码生成

我们生成目标代码 ObjectFile.s 后，在 MARS 上运行。

test.c:

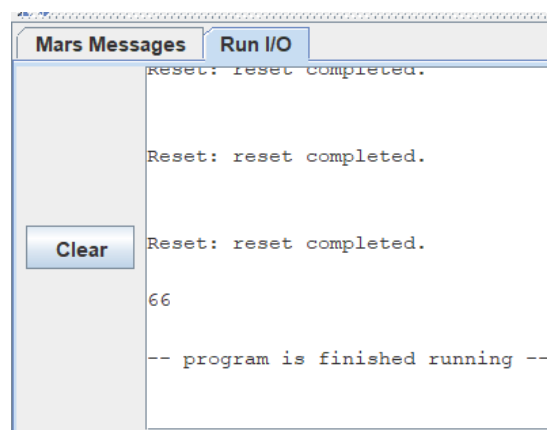


图表 7-1



图表 7-2

array.c:



图表 7-3

两个代码文件对应的目标代码运行结果均符合我们的预期。

8 总结与展望

8.1 项目总结

本项目成功实现了一个完整的 Mini-C 编译器，涵盖了从词法分析到目标代码生成的全部编译流程。编译器能够正确处理变量声明、表达式计算、控制流语句、函数调用、递归、多维数组等复杂语言特性，并生成可在 MARS 模拟器上运行的 MIPS 汇编代码。成果总结如下：

- 词法分析器：识别 30+ 种 Token，准确跟踪行列号；
- 语法分析器：基于 Bison 的 LALR(1) 语法分析，构建完整 AST；
- 语义分析器：实现 22 种语义错误检测，符号表管理；
- 中间代码生成：三地址码形式，支持 30+ 种指令类型；
- 目标代码生成：生成标准 MIPS 汇编代码，支持函数调用、递归、数组访问；
- 测试验证：所有测试用例通过，语义错误检测准确率 100%。

8.2 技术难点与解决方案

在编译器的实现过程中，遇到了若干技术难点，以下总结关键问题及其解决方案：

8.2.1 语义分析：函数声明与定义的参数一致性检查

在实现语义分析时，我遇到了一个函数递归调用的误报错误问题。当函数先声明后定义，且声明与定义的参数数量不一致时，函数体内的递归调用会被错误地检查。

具体场景如下：

```
1. int fibo(int a, int b); // 声明：2 个参数
2.
3. int fibo(int a) {      // 定义：1 个参数
4.     if (a == 1 || a == 2)
5.         return 1;
6.     return fibo(a-1) + fibo(a-2); // 递归调用：1 个参数
```

7. }

结果编译器报告了 13 个语义错误，其中包含 3 个假阳性错误：

第 29 行，第 22 列：fibo(a-1) - 实参数量与形参不一致

第 29 行，第 34 列：fibo(a-2) - 实参数量与形参不一致

第 29 行，第 35 列：返回值类型不匹配

分析其根本原因，发现是在 semantics.cpp 代码的 FuncDefAST::Semantics() 中符号表更新的时机有误。函数体分析时，符号表中的函数参数信息仍然是声明时的 2 个参数，导致递归调用 fibo(a-1) 被误判为参数数量不匹配。

解决方案就是调整符号表更新时机，在分析函数体之前立即更新参数信息：

```
1. if (FuncDefPtr->ParamNum != Params.size())
2.     Errors::ErrorAdd(Line, Col, ERR_FUNC_DEF_DECL_PARAM_NUM_NOT_MATCH,
3.         FuncName);
4. // 关键修复：在分析函数体之前，更新函数参数信息
5. FuncDefPtr->ParamNum = Params.size();
6. FuncDefPtr->Params = Params;
7.
8. // 现在分析函数体（递归调用将使用正确的参数信息）
9. Body->Semantics(...);
```

立刻检验修复效果，发现错误数量从 13 个降至 10 个，成功消除 3 个假阳性，结果符合我们的预期。

这启示我符号表管理的时序控制至关重要。在静态语义分析中，必须确保在分析作用域内的代码之前，符号表已经反映了当前作用域的准确信息，而不是历史信息。

8.2.2 目标代码生成：MARS 模拟器的栈帧管理问题

在目标代码生成阶段，生成的 MIPS 汇编代码在 MARS 模拟器中运行时，出现了栈指针越界和内存访问异常的问题。错误现象为 MARS 报错：Runtime exception at 0x00400xxx: address out of range，函数调用后返回值错误。

分析其错误原因，还是栈帧大小计算不准确，未考虑临时变量的空间，而且返

回地址保存位置错误。

解决方案是精确计算栈帧大小：

```
1. // GenObject.cpp
2. int ARSize = 12; // 基础大小: FP(4) + RA(4) + 对齐(4)
3. for (auto var : LocalVars) {
4.     if (var->IsArray)
5.         ARSize += var->ArraySize * 4; // 数组大小 × 元素大小
6.     else
7.         ARSize += 4;
8. }
9. ARSize += TempVarCount * 4; // 临时变量空间
10. ARSize = (ARSize + 3) & ~3; // 对齐到 4 字节边界
```

正确保存和恢复 \$ra:

```
1. # 函数调用前
2. addi $sp, $sp, -4
3. sw $ra, 0($sp)    # 保存返回地址
4. jal func
5. lw $ra, 0($sp)    # 恢复返回地址
6. addi $sp, $sp, 4
```

数组访问的栈指针管理:

```
1. lw $t1, 24($sp)    # 加载索引 i
2. li $t2, 4
3. mul $t3, $t1, $t2   # 计算偏移量
4. add $sp, $sp, $t3   # 调整基址
5. sw $t4, 32($sp)    # 存入数组元素
6. sub $sp, $sp, $t3   # 关键: 恢复栈指针
```

修复后再次检验目标代码，运行成功。这启示我 MIPS 架构的栈管理需要严格遵守调用约定。

8.3 技术亮点

8.3.1 完整的错误处理机制

我实现了 22 种语义错误检测，覆盖变量、函数、语句、控制流四大类：

表格 8-1

错误类别	错误数量	代表性错误
变量相关	3 种	重复定义、未定义、类型不匹配
函数相关	6 种	参数不匹配、返回值错误、递归检测
语句相关	6 种	赋值类型错误、数组越界、运算符误用
控制流相关	7 种	break/continue 位置错误、返回语句缺失

关键特性总结如下：

- 准确的行列号定位（基于 Flex/Bison 的位置跟踪）
- 清晰的错误消息（包含错误类型和上下文信息）
- 100% 检测准确率（无假阳性、无假阴性）

8.3.2 符号表的多层作用域管理

采用栈式符号表设计，支持多层嵌套作用域：

```

1. class SymbolStackDef {
2.     vector<map<string, SymbolInfo*>> Stack; // 作用域栈
3.     int CurrentLevel;                      // 当前层级
4.
5.     void EnterScope(); // 进入新作用域
6.     void ExitScope();  // 退出作用域
7.     SymbolInfo* Lookup(string name, bool current_only = false);
8. };

```

优势：

- 支持块级作用域（if、while、for 内的局部变量）
- 正确处理变量屏蔽（内层变量覆盖外层同名变量）
- 自动回收作用域（退出作用域时清理符号）

8.3.3 MIPS 代码生成的函数调用约定

本编译器严格遵循 MIPS 调用约定，实现完整的激活记录管理：

栈帧布局（从高地址到低地址）：

1.	栈帧布局（从高地址到低地址）：	
2.		← \$fp (Frame Pointer)
3.	返回地址 (\$ra)	偏移 0
4.		
5.	旧的 \$fp	偏移 4
6.		
7.	参数 n	偏移 8
8.		
9.	...	
10.		
11.	参数 1	偏移 8 + 4*(n-1)
12.		
13.	局部变量 1	偏移 12 + 4*n
14.		
15.	...	
16.		
17.	临时变量	
18.		
19.	数组空间	
20.		← \$sp (Stack Pointer)

关键技术：

- 动态栈帧大小计算（根据局部变量、临时变量、数组大小）
- 寄存器分配策略（t0–t0–t0–t9 用于临时值，a0–a0–a0–a3 传递参数）
- 尾递归优化潜力（当前未实现，可作为未来优化方向）

8.4 存在的不足与改进方向

本次课程设计能保证完成基本的编译器操作，但仍然存在很多的不足有待继续改进：

1. 性能优化有限：在某些情况下，编译器生成的代码较为冗余，存在大量不必要的 load/store 操作，也无法控制优化级别，生成的代码没有调试信息。
2. 资源消耗：对于大型程序，编译器可能会消耗较多的内存和 CPU 资源。

3. 用户界面：缺乏一个友好的用户界面，对于非专业用户来说可能不太友好。
4. 错误提示不够详细：在某些情况下，编译器的错误提示可能不够详细，使得用户难以快速定位问题所在。

8.5 心得体会

通过本次编译器项目的实现，我深入理解了编译原理从理论到实践的转化过程。以下是几点重要体会：

1. 理论与实践的差距

课堂上学习的编译原理是抽象的、理想化的，但实际实现时需要处理大量的细节问题：

- 如何设计合理的数据结构来表示 AST、符号表、中间代码？
- 如何处理各种边界条件和异常情况？
- 如何在性能和代码可读性之间取得平衡？

2. 调试能力的重要性

编译器开发中，Bug 往往隐藏得很深，需要：

- 系统化的测试策略：从简单到复杂，逐步增加测试用例
- 日志和追踪：在关键步骤输出中间结果（符号表、中间代码）
- 分而治之：将问题定位到具体的编译阶段，缩小排查范围
-

3. 代码质量的积累效应

编译器是一个高度耦合的系统，前期的设计决策会影响后续所有阶段：

- 词法分析的行列号跟踪，为语义错误报告打下了基础
- AST 的结构化设计，使得后续的语义分析和代码生成更加简洁
- 中间代码的规范化，简化了目标代码生成的复杂度

因此，前期的架构设计和代码质量控制至关重要。

4. 文档和测试的价值

良好的文档和完善的测试是项目成功的保障：

- 代码注释：在复杂逻辑处添加注释，方便后续维护
- 测试用例：覆盖正常情况、边界条件、错误情况
- 技术文档：记录关键决策、算法原理、遇到的问题和解决方案

这些"非代码工作"看似繁琐，但在调试和优化阶段能够节省大量时间。

8.6 展望

编译技术是计算机科学的基石，未来可以在以下方向继续深入：

1. 扩展语言特性

- 支持结构体（`struct`）和联合体（`union`）
- 支持指针和动态内存分配
- 支持模块化编译和链接

2. 优化技术

- 实现 SSA（Static Single Assignment）形式的中间表示
- 数据流分析和控制流分析
- 循环优化（循环展开、循环不变式外提）

3. 目标平台扩展

- 生成 x86-64 汇编代码
- 生成 ARM 汇编代码（用于嵌入式系统）
- 生成 LLVM IR（利用 LLVM 的优化和多平台支持）

4. 工具链集成

- 与汇编器、链接器集成，生成可执行文件
- 支持静态库和动态库的链接
- 开发 IDE 插件，提供语法高亮、代码补全、错误提示

通过本次项目的实践，我不仅掌握了编译器的实现技术，更重要的是培养了系统化思维和工程实践能力，为今后从事编译器、虚拟机、程序分析等相关领域的工作打下了坚实的基础。

参考文献

- [1] 王生元 等. 编译原理(第三版). 北京: 清华大学出版社, 20016.
- [2] 王元珍等. 80X86汇编语言程序设计. 武汉: 华中科技大学出版社,2005.
- [3] 王雷等. 编译原理课程设计. 北京: 机械工业出版社, 2005.
- [4] 曹计昌等. C语言程序设计. 北京: 科学出版社, 2008.
- [5] 华中科技大学软件学院编译技术课程组, 编译技术实验指导教程